

## CSC3109 MACHINE LEARNING

SINGAPORE INSTITUTE OF TECHNOLOGY

INFOCOMM TECHNOLOGY

---

# Group 23: A Controlled Comparison of Five Deep Learning Architectures for Fine-Grained Aerial Scene Classification

---

*Students:*

Pang Jia De (ID: 2402226, 3070684P)

Daryl Chong Han Kin (ID: 2400619, 3070590C)

Kaiser Yee (ID: 2400821, 3070609Y)

Tan Jian Xin (ID: 2401421, 3070651T)

Jun Kiat (ID: 2403339, 3070561T)

Date: July 25, 2026

## Contents

|          |                                                            |           |
|----------|------------------------------------------------------------|-----------|
| <b>1</b> | <b>Introduction</b>                                        | <b>4</b>  |
| 1.1      | Background and Motivation . . . . .                        | 4         |
| 1.2      | Problem Statement . . . . .                                | 4         |
| 1.3      | Project Objectives . . . . .                               | 5         |
| 1.4      | Review of Existing Approaches . . . . .                    | 6         |
| 1.4.1    | From Handcrafted Descriptors to Learned Features . . . . . | 6         |
| 1.4.2    | Benchmark Datasets . . . . .                               | 6         |
| 1.4.3    | Convolutional Architectures . . . . .                      | 7         |
| 1.4.4    | Transfer Learning under Limited Supervision . . . . .      | 7         |
| 1.4.5    | Attention and Vision Transformers . . . . .                | 7         |
| 1.4.6    | Positioning of This Work . . . . .                         | 8         |
| 1.4.7    | Open-Source Libraries Used . . . . .                       | 8         |
| <b>2</b> | <b>Exploratory Data Analysis</b>                           | <b>8</b>  |
| 2.1      | Class Balance . . . . .                                    | 9         |
| 2.2      | Image Properties . . . . .                                 | 9         |
| 2.3      | Colour Statistics . . . . .                                | 9         |
| 2.4      | Visual Confusability . . . . .                             | 10        |
| 2.5      | Data Quality: Duplicate Images . . . . .                   | 10        |
| 2.6      | Implications for Modelling . . . . .                       | 11        |
| <b>3</b> | <b>Data Preparation and Pre-Processing</b>                 | <b>11</b> |
| 3.1      | Data Splits and Leakage Control . . . . .                  | 11        |
| 3.2      | Input Resolution . . . . .                                 | 12        |
| 3.3      | Augmentation . . . . .                                     | 12        |
| <b>4</b> | <b>Deep Learning Approaches</b>                            | <b>14</b> |
| 4.1      | Custom CNN (from scratch) . . . . .                        | 14        |
| 4.1.1    | Source Architecture . . . . .                              | 14        |
| 4.1.2    | Training Configuration . . . . .                           | 15        |
| 4.1.3    | Results . . . . .                                          | 16        |
| 4.1.4    | Discussion . . . . .                                       | 18        |
| 4.2      | ResNet-50 (transfer learning) . . . . .                    | 19        |
| 4.2.1    | Results . . . . .                                          | 20        |
| 4.3      | EfficientNet-B0 (transfer learning) . . . . .              | 21        |
| 4.3.1    | Architecture and Adaptation . . . . .                      | 22        |
| 4.3.2    | Position against Published Architectures . . . . .         | 22        |
| 4.3.3    | Results . . . . .                                          | 23        |
| 4.4      | MobileNetV2 (transfer learning) . . . . .                  | 24        |
| 4.4.1    | Architecture and Adaptation . . . . .                      | 25        |
| 4.4.2    | Position against Published Mobile Models . . . . .         | 25        |
| 4.4.3    | Results . . . . .                                          | 26        |
| 4.5      | Vision Transformer (ViT) . . . . .                         | 27        |
| 4.5.1    | Results . . . . .                                          | 29        |

---

|           |                                                                              |           |
|-----------|------------------------------------------------------------------------------|-----------|
| <b>5</b>  | <b>Model Training and Tuning</b>                                             | <b>31</b> |
| 5.1       | Shared Training Protocol . . . . .                                           | 31        |
| 5.2       | Hyper-parameter Tuning . . . . .                                             | 32        |
| 5.3       | Convergence . . . . .                                                        | 32        |
| <b>6</b>  | <b>Inference and Performance Evaluation</b>                                  | <b>33</b> |
| 6.1       | Evaluation Protocol . . . . .                                                | 33        |
| 6.2       | Results . . . . .                                                            | 33        |
| <b>7</b>  | <b>Strengths, Weaknesses, and Comparative Insights</b>                       | <b>34</b> |
| 7.1       | Per-Approach Assessment . . . . .                                            | 34        |
| 7.2       | Accuracy versus Cost . . . . .                                               | 36        |
| 7.3       | Data Efficiency: What Pretraining Buys . . . . .                             | 36        |
| 7.4       | Capacity and the Trustworthiness of Validation Metrics . . . . .             | 36        |
| 7.5       | Error Overlap and Diversity . . . . .                                        | 37        |
| <b>8</b>  | <b>Deployment via Containerisation</b>                                       | <b>37</b> |
| 8.1       | Deployed Model . . . . .                                                     | 37        |
| 8.2       | Container Architecture . . . . .                                             | 37        |
| 8.3       | Inference Pipeline . . . . .                                                 | 38        |
| 8.4       | Design Decisions . . . . .                                                   | 38        |
| 8.5       | Build and Deployment . . . . .                                               | 39        |
| 8.6       | Verification and Limitations . . . . .                                       | 39        |
| <b>9</b>  | <b>Results, Insights, and Discussion</b>                                     | <b>40</b> |
| <b>10</b> | <b>Individual Contributions</b>                                              | <b>41</b> |
| <b>11</b> | <b>Reflection</b>                                                            | <b>42</b> |
| 11.1      | Evaluation Discipline Is a Design Problem, Not a Willpower Problem . . . . . | 42        |
| 11.2      | A Controlled Comparison Is a Team Contract . . . . .                         | 42        |
| 11.3      | Transfer Learning in a Low-Data Regime . . . . .                             | 43        |
| 11.4      | Engineering Lessons from the Debugging Trail . . . . .                       | 43        |
| 11.5      | What We Would Do Differently . . . . .                                       | 44        |
| <b>12</b> | <b>Future Work</b>                                                           | <b>44</b> |
| <b>A</b>  | <b>Supplementary Material</b>                                                | <b>47</b> |
| A.1       | Consolidated Hyper-parameters . . . . .                                      | 47        |
| A.2       | Repository Structure . . . . .                                               | 47        |
| A.3       | Reproduction Instructions . . . . .                                          | 48        |

---

# 1 Introduction

## 1.1 Background and Motivation

Earth-observation satellites, aerial surveys, and high-resolution basemaps now produce far more imagery than analysts can review by hand [15]. Extracting useful information from that imagery — such as the locations of infrastructure and changes over time — is important for urban planning, environmental monitoring, infrastructure management, and disaster response. Scene classification, labelling an image patch with a land-use category, is a foundational task for many of these applications.

Until the early 2010s this problem was tackled with handcrafted features: colour histograms, texture descriptors, SIFT-based bag-of-visual-words. These worked reasonably well for coarse distinctions but stalled because they describe low-level appearance rather than the structure of a scene. Deep convolutional networks changed that by learning features directly from data instead of relying on hand-designed ones. Since AlexNet’s ImageNet result [7], they have taken over image classification generally, and the remote-sensing literature shows the same pattern: deep models are now standard on aerial scene classification benchmarks [3].

The dataset used in this project makes the task harder in two specific ways. First, the four classes — `coastal_mansion`, `dense_residential`, `nursing_home`, and `sparse_residential` — all look visually similar to each other when seen from above. What separates them is density and spatial arrangement rather than distinct objects or colours, so a model has to recognise layout rather than merely the presence of objects. Second, there are only 700 training images per class, which is little enough that a network trained from scratch would likely overfit. This is why data augmentation and transfer learning from pretrained models are central to our approach rather than optional extras.

We also cared about two things beyond accuracy. One is fair comparison: architecture papers are often hard to compare because they differ in data splits, augmentation, training budgets, or evaluation setup, so we fixed those factors across our experiments and documented the few architecture-specific differences (preprocessing, effective backbone resolution, and the custom CNN’s learning-rate schedule). The other is deployment; a model is useful only if it can be deployed in a target environment. We therefore packaged the best-performing model as a containerized inference service rather than leaving it as a notebook result.

## 1.2 Problem Statement

Each team in this cohort is assigned a partition of a 38-category aerial-imagery benchmark. The class names, the 800 images per class, and the native  $256 \times 256$  pixel resolution identify the source as **PatternNet**, a high-resolution dataset collected mainly over US cities from Google Earth and Google Map API imagery. Our partition (*set 23*) contains four categories:

`coastal_mansion`, `dense_residential`, `nursing_home`, `sparse_residential`,

each with 700 training images and 100 held-out validation images. The held-out images never appear in training and are used only for final evaluation.

The task is standard four-way image classification: train a model that takes a  $256 \times 256$  RGB aerial image and outputs a probability for each of the four classes, and evaluate it on the held-

out set using accuracy, per-class and macro-averaged precision, recall, and  $F_1$ -score, and the confusion matrix.

What makes this partition hard is that all four categories are residential or institutional building scenes. A nursing home is essentially a large residential complex; dense and sparse residential areas differ in lot size and building spacing rather than in kind; the most reliable cue for a coastal mansion is the shoreline next to it, not the building itself. So the classes look alike (same mix of buildings, roads, and vegetation) while images within a class vary a lot (roof colour, vegetation cover, season, layout).



**Figure 1:** Representative samples from each class in our partition (set 23) of PatternNet. All four categories share buildings, roads, and vegetation as their basic visual vocabulary; the distinguishing cues are density, spacing, and, for coastal mansions, shoreline adjacency rather than the built structures themselves.

The project specification adds three practical requirements: (i) the team must investigate at least as many deep learning approaches as it has members (five); (ii) every open-source library used must be registered and justified, and our own contributions documented; and (iii) the best model must be deployed in a container that accepts an aerial image and returns a predicted label with per-class softmax scores.

### 1.3 Project Objectives

We set ourselves six objectives:

1. **O1: Understand the data.** Explore class balance, image statistics, and visual overlap between categories, and use the findings to justify pre-processing and augmentation choices.
2. **O2: Evaluate without leakage.** Use a shared data pipeline with a fixed internal tuning split for development and checkpoint selection, and touch the 400 held-out images only once, for final evaluation.

3. **O3: Investigate five approaches.** One per team member: a from-scratch CNN baseline, three pretrained CNN families (ResNet-50, EfficientNet-B0, MobileNetV2), and a vision transformer.
4. **O4: Compare fairly.** Train every model with the same seed, dataset split, raw input size, augmentation, batch size, epoch budget, and early-stopping patience, while explicitly documenting necessary architecture-specific preprocessing and optimisation choices.
5. **O5: Maximise held-out performance.** Get the best accuracy and macro- $F_1$  we can, and use confusion matrices to understand the remaining errors.
6. **O6: Deploy the best model.** Package it in a Docker container with a web interface that returns the predicted category and per-class softmax scores, with automated contract tests covering the shared data pipeline and training endpoint.

## 1.4 Review of Existing Approaches

### 1.4.1 From Handcrafted Descriptors to Learned Features

Early remote-sensing scene classification relied on handcrafted descriptors — colour histograms, texture measures, and SIFT-based bag-of-visual-words representations. These encode low-level appearance and therefore struggle precisely where our partition is hardest: distinguishing categories that share the same visual vocabulary and differ only in spatial arrangement. The shift to learned representations after AlexNet [7] resolved this by inferring features directly from data, and the remote-sensing literature has followed the same trajectory [15, 3].

The magnitude of the improvement is well documented. In an aerial forest segmentation experiment, Kwenda et al. [8] report 0.646 accuracy for a Random Forest operating without deep features and 0.940 when the same classifier uses a concatenated feature vector extracted by VGG16 and ResNet-50. The paper describes the two networks as feature extractors but does not state that they were frozen throughout. Zhou et al. [14] report the same broad ordering on the benchmark used in this project, where CNN-derived features outperform handcrafted descriptors for retrieval. Both results motivate our decision to treat deep feature extraction, rather than feature engineering, as the foundation of every approach we investigate.

### 1.4.2 Benchmark Datasets

Progress in this field has been driven by successive benchmarks — UC Merced, WHU-RS19, RSSCN7, AID, and NWPU-RESISC45 — each larger or more finely categorised than the last. Our data comes from **PatternNet** [14], a 38-class collection of 800 images per class at  $256 \times 256$  pixels, gathered over US cities via Google Earth imagery and the Google Map API.

One design property of PatternNet directly shapes this project. The dataset was constructed for *image retrieval*, a task that rewards fine-grained category distinctions, so it deliberately separates classes that a coarser-grained benchmark would merge. Our partition inherits four such categories — `coastal_mansion`, `dense_residential`, `nursing_home`, and `sparse_residential` — all of which are residential or institutional building scenes. The difficulty we report in Section 6.2 is therefore intrinsic to the benchmark’s design rather than incidental to our partition.

### 1.4.3 Convolutional Architectures

The systematic review by Shoaib et al. [10], covering 124 studies of deep learning for aerial imagery, identifies convolutional networks as the dominant family for classification, valued for accuracy and computational efficiency. Three developments in that family inform our choices.

**Residual learning.** ResNet [5] introduced skip connections so that each block learns a residual adjustment to its input, allowing gradients to propagate through very deep stacks. ResNet-50 remains a standard baseline with well-understood transfer behaviour, which is why we include it.

**Compound scaling.** EfficientNet [12] observes that network depth, width, and input resolution should be scaled together under a single coefficient rather than independently, yielding markedly better accuracy per parameter. Shoaib et al. [10] single out this efficiency as important for high-resolution aerial data, where computational cost scales quickly with image size.

**Efficiency-oriented design.** MobileNetV2 [9] factorises standard convolutions into depthwise and pointwise operations and adds inverted residual blocks with linear bottlenecks, reducing parameter count by an order of magnitude at modest accuracy cost. It represents the deployment-constrained end of the design space, and the development of such lightweight models is identified as a priority direction in [10].

### 1.4.4 Transfer Learning under Limited Supervision

With 700 training images per class, training a large network from random initialisation risks overfitting. The standard remedy is transfer learning: reusing representations learned on a large source corpus such as ImageNet. Shoaib et al. [10] state the case directly — pretrained models “provide robust feature representations that generalize well to aerial imagery,” and this “is especially important for scenarios with limited labeled data.”

The practical question is how much of the network to adapt. Kwenda et al. [8] take the conservative extreme, freezing VGG16 and ResNet-50 entirely and training only a downstream classifier, and still obtain strong results. We adopt the same strategy for our three pretrained models: the backbone is frozen and only a small classification head is fitted. This suppresses overfitting on a small dataset and, equally important for our purpose, holds the amount of adaptation constant across the transfer-learning approaches so that the comparison reflects architecture rather than fine-tuning effort.

### 1.4.5 Attention and Vision Transformers

Vision Transformers [4] discard convolution entirely, partitioning an image into fixed patches and processing the resulting sequence with self-attention. Every patch can attend to every other from the first layer, capturing global spatial layout that a convolutional stack only reaches after many layers — a property well matched to aerial scenes, where the discriminating cue is the arrangement of elements across the whole image.

This capability carries a known cost. Shoaib et al. [10] caution that transformers “typically require large amounts of training data and computational resources, which can be a limitation in domains like aerial image analysis with limited annotated datasets.” Whether that limitation binds at our dataset scale is one of the questions this study answers empirically (Section 7).

### 1.4.6 Positioning of This Work

The literature establishes that deep features outperform handcrafted ones, that transfer learning is the standard response to limited data, and that architectures differ in their accuracy-to-cost trade-offs. What it does not readily provide is a *controlled* comparison: published results differ in input resolution, augmentation, optimiser, training schedule, and evaluation protocol, so architectural claims are difficult to isolate. This project addresses that gap on one dataset by fixing every such variable across five architectures and varying only the model, as detailed in Section 3.1 and Section 3.3.

### 1.4.7 Open-Source Libraries Used

Table 1 registers every third-party component used in this project. All are open-source and used under their respective licences; no pretrained checkpoint or code was used outside its intended terms.

**Table 1:** Open-source libraries and tools, with the role each plays.

| Library            | Version      | Role in this project                                           |
|--------------------|--------------|----------------------------------------------------------------|
| TensorFlow / Keras | $\geq 2.15$  | Model definition, training, augmentation layers, checkpointing |
| keras-hub          | $\geq 0.3.0$ | Pretrained Vision Transformer backbone                         |
| scikit-learn       | $\geq 1.3$   | Precision, recall, $F_1$ , and confusion-matrix computation    |
| NumPy              | $\geq 1.24$  | Array operations and metric aggregation                        |
| pandas             | $\geq 2.0$   | Tabular summaries during exploratory analysis                  |
| Matplotlib         | $\geq 3.7$   | Figures: sample grids, training curves, confusion matrices     |
| Pillow             | $\geq 10.0$  | Image loading and inspection for EDA                           |
| pytest             | $\geq 8.0$   | Automated contract tests for the data and training pipelines   |
| Streamlit          | 1.58.0       | Web interface for the deployed classifier                      |
| Docker             | —            | Containerisation of the deployed model (python:3.12-slim)      |

Pretrained ImageNet weights for ResNet-50, EfficientNet-B0, and MobileNetV2 are obtained through the Keras applications module; the Vision Transformer backbone is obtained through keras-hub. All model definitions, the shared data pipeline, the augmentation policy, the training and evaluation entrypoints, the automated tests, and the deployment application were written by the team; our specific contributions and modifications are documented in Section 10.

## 2 Exploratory Data Analysis

Exploratory analysis was carried out before any model was built, and deliberately restricted to the training split. Only the *counts* of the held-out directory were inspected; no held-out image was viewed or measured, so none of the design decisions that follow could have been informed by the data used for final evaluation.

## 2.1 Class Balance

The dataset is exactly balanced. Each of the four categories contributes 700 training images and 100 held-out images, giving 2,800 and 400 respectively (Table 2).

**Table 2:** Class distribution across the provided splits.

| Class              | Train | Held-out | Total |
|--------------------|-------|----------|-------|
| coastal_mansion    | 700   | 100      | 800   |
| dense_residential  | 700   | 100      | 800   |
| nursing_home       | 700   | 100      | 800   |
| sparse_residential | 700   | 100      | 800   |
| Total              | 2,800 | 400      | 3,200 |

Perfect balance has two consequences for the rest of this report. First, plain accuracy is not misleading here, because a majority-class predictor is capped at 25%; we nevertheless report macro- $F_1$  alongside it, since macro-averaging exposes a collapse on any single class that accuracy alone can hide. Second, no class weighting or resampling is required, which removes a confounding variable from the five-model comparison.

## 2.2 Image Properties

A full file audit of all 3,200 images across the provided training and held-out directories confirmed that every image is  $256 \times 256$  pixels, has an aspect ratio of exactly 1.0, and uses RGB mode. No greyscale, CMYK, alpha-channel, or corrupt images were encountered. The audit is retained as an automated data-contract test so the claim can be rechecked from the repository.

This uniformity motivated a concrete team decision. The pipeline was originally configured at the conventional ImageNet resolution of  $224 \times 224$ , but because the source images are natively  $256 \times 256$ , that setting discarded roughly 23% of the available pixels before training began. The shared external input was therefore standardised at `IMAGE_SIZE = 256`. The four CNNs operate on the data at that native resolution; ViT accepts the same  $256 \times 256$  tensor and then resizes it internally to  $224 \times 224$  to match its fixed position-embedding grid. The trade-off is revisited in Section 7.

## 2.3 Colour Statistics

Mean per-channel intensities were computed over 60 images per class (Table 3). The striking result is how little colour separates the categories: all twelve channel means fall within a narrow band of roughly 60–100 intensity units.

**Table 3:** Mean per-channel intensity by class (60 training images per class).

| Class              | R    | G     | B    |
|--------------------|------|-------|------|
| coastal_mansion    | 74.2 | 71.6  | 63.0 |
| dense_residential  | 89.7 | 86.3  | 77.3 |
| nursing_home       | 96.9 | 100.6 | 88.0 |
| sparse_residential | 70.4 | 74.6  | 60.7 |

`coastal_mansion` and `sparse_residential` are especially close, differing by under 4 intensity units in every channel — both are low-density scenes dominated by vegetation and large plots. `nursing_home` is the brightest class, consistent with large institutional roofs. The practical implication is that a classifier cannot succeed on colour alone: the discriminative signal lies in *texture and spatial structure* — building density, roof arrangement, road layout, and the regularity of the built pattern. This prediction is borne out by the results in Section 6.2, where the residual errors are concentrated in exactly the density-based distinctions rather than colour-based ones.

## 2.4 Visual Confusability

Inspecting sample grids per class confirms that this is a genuinely fine-grained problem rather than a coarse one. All four categories are suburban aerial scenes photographed at similar altitude, sharing the same vocabulary of roofs, roads, and vegetation; they differ only in the *arrangement* and *density* of those elements. Two pairs are particularly difficult:

- `dense_residential` versus `sparse_residential` — an identical scene type separated only by a continuous, unlabelled density threshold, with no discrete visual boundary between them.
- `nursing_home` versus `dense_residential` — both show clustered multi-unit buildings; the distinction rests on building footprint and layout regularity rather than on scene content.

These predictions were recorded before training and are directly confirmed by the confusion matrices in Section 6.2.

## 2.5 Data Quality: Duplicate Images

One integrity problem was found. Two training files are byte-identical to each other yet assigned to conflicting classes:

```
coastal_mansion/coastalmansion029.jpg  and  
sparse_residential/sparseresidential021.jpg
```

The same image therefore carries two mutually exclusive labels. Left in place it supplies contradictory gradients and, worse, could place identical pixels on both sides of the train/tuning split — a small but genuine leakage path. Both files are excluded logically by the shared loader through `EXCLUDED_TRAIN_FILES` in `shared/config.py`; the source data on disk is never modified, so the exclusion is reproducible and auditable by every team member. The usable training pool is consequently 2,798 images. Notably, the affected pair belongs to the two classes that colour statistics already identified as most similar.

## 2.6 Implications for Modelling

Four conclusions carried forward into the design:

1. **Accuracy is a fair headline metric** (balanced classes), but macro- $F_1$  is reported alongside it to expose per-class collapse.
2. **All models accept  $256 \times 256$  inputs.** The four CNNs retain the native resolution; ViT alone performs a documented internal resize to  $224 \times 224$ .
3. **Texture and structure, not colour, carry the signal**, so the augmentation policy must preserve fine-grained texture — photometric jitter is kept deliberately mild for this reason (Section 3.3).
4. **Aerial scenes have no canonical orientation.** Unlike natural photographs, a top-down scene is equally valid flipped or rotated, so full horizontal and vertical flips and arbitrary rotations are *label-preserving* here. This observation directly justifies the shared augmentation policy.

## 3 Data Preparation and Pre-Processing

### 3.1 Data Splits and Leakage Control

A recurring flaw in student-scale deep learning experiments is that the same data guides both model selection and final reporting, which inflates the headline number without anyone intending to cheat. We designed the pipeline to reduce that risk through a three-way split implemented once in `shared/data.py` and shared by all five models.

**Three-way split.** The provided `val_23` directory is treated as a *sealed test set*. All development data is therefore carved out of `set_23` alone: a deterministic stratified split assigns 85% of it to model fitting and 15% to an internal *tuning* set (Table 4).

**Table 4:** Three-way data split. Only the training subset updates weights; the tuning subset guides checkpoint selection; the held-out set is untouched until final evaluation.

| Subset   | Source              | Images | Role                                 |
|----------|---------------------|--------|--------------------------------------|
| Training | <code>set_23</code> | 2,378  | gradient updates                     |
| Tuning   | <code>set_23</code> | 420    | early stopping, checkpoint selection |
| Held-out | <code>val_23</code> | 400    | final evaluation only                |

The split is *stratified*: it is drawn independently within each class, so the tuning subset contains exactly 105 images per class and the training subset 594–595 per class. Class balance is thus preserved on both sides rather than left to chance, which matters because per-class  $F_1$  on a 420-image subset is sensitive to even small imbalances.

**Determinism.** Split repeatability is supported by seeding the shuffle for class  $k$  with  $\text{SEED} + k$  ( $\text{SEED} = 42$ ) rather than by a global random state. Consequently the partition depends on the file listing rather than execution order or earlier random draws. Given the same dataset snapshot and software contract, team members should derive the same subsets. The exact counts are recorded in each run’s metadata via `get_split_counts()`, so any run can be audited after the fact.

**Why the held-out set must stay sealed.** Both `ModelCheckpoint` and `EarlyStopping` select the epoch that maximises *tuning* accuracy. That makes tuning metrics optimistically biased by construction: they measure the best of roughly 30 attempts, not the expected performance on new data. The effect is not hypothetical — for our custom CNN the tuning macro- $F_1$  of 0.908 overstates the held-out 0.845 by more than six points. Every number reported in Section 6.2 is therefore held-out, and the tuning metrics are retained only as a training diagnostic. Mechanically, the held-out loader is a separate function that reads `val_23` and is invoked only behind an explicit `--evaluate-held-out` flag, after training has finished. It is never passed to `model.fit` and never influences a callback. Combined with the duplicate-image exclusion described above, this design keeps exact duplicate pixels out of both the training and held-out evaluation paths under the shared entripoint.

**Deviation from the literature protocol.** Our effective proportions are approximately 74/13/13 rather than the 70/15/15 used by the reference architecture we follow, because the test partition is fixed at 400 images by the provided data rather than chosen by us. Since the identical split is applied to all five models, the split choice is controlled across approaches even though its sampling uncertainty still affects the comparison.

## 3.2 Input Resolution

All models accept  $256 \times 256$  RGB inputs, the *native* resolution of every source image. The four CNNs retain that spatial resolution. ViT is the documented exception: its saved graph applies bilinear resizing from  $256 \times 256$  to  $224 \times 224$  immediately before the backbone because its position embeddings are fixed to a  $14 \times 14$  patch grid. Standardising the external interface at 256 therefore gives every approach the same input contract, but does not imply that every architecture preserves every source pixel internally.

The trade-off is that the ImageNet-pretrained backbones no longer operate at the resolution they were trained on. We accepted this deliberately: the alternative penalises the from-scratch model (which has no such prior) in order to flatter the pretrained ones, and comparability requires that all five approaches see identical inputs. Resizing uses bilinear interpolation with antialiasing enabled, which matters when a model internally rescales.

## 3.3 Augmentation

With only 2,378 training images against models of 2.3M–85.8M parameters, augmentation is not an optional refinement but the main defence against overfitting. The policy is defined once in `shared/augment.py` and embedded as the first stage of every model, so it is applied identically across all five approaches and is automatically inactive at inference time.

**Table 5:** Shared training-time augmentation policy. All layers are seeded for reproducibility and operate on raw  $[0, 255]$  pixels.

| Transform         | Setting                        | Justification                      |
|-------------------|--------------------------------|------------------------------------|
| Random flip       | horizontal + vertical          | no canonical orientation           |
| Random rotation   | $\pm 180^\circ$ , reflect fill | no canonical orientation           |
| Random zoom       | $\pm 20\%$                     | capture altitude / scale variation |
| Random contrast   | $\pm 10\%$                     | sensor and lighting variation      |
| Random brightness | $\pm 10\%$                     | sensor and lighting variation      |

**Geometric transforms are label-preserving here.** For natural photographs a vertical flip produces an implausible image, and practitioners routinely restrict augmentation to horizontal flips and small rotations for that reason. Top-down aerial imagery has no such constraint: a residential block photographed from above is equally valid rotated by any angle or mirrored in either axis, and the class label is unchanged. Full flips and arbitrary rotations are therefore *free* additional training signal rather than label noise, and we exploit them aggressively. Rotation uses `reflect` fill so that rotated corners contain plausible texture instead of black wedges, which would otherwise be a spurious cue correlated with augmentation rather than with class.

**Photometric transforms are kept deliberately mild.** Contrast and brightness are restricted to  $\pm 10\%$ , making the photometric policy deliberately milder than the full flips and rotations used for the label-preserving geometric policy. This asymmetry follows from the exploratory finding that the four classes are *not* separable by colour: with all twelve channel means confined to a narrow band, the discriminative signal lies in fine-grained texture and spatial structure. Aggressive photometric jitter would degrade precisely the cues the models depend on. This was confirmed empirically — an early configuration using a  $\pm 50\%$  factor drove images to near-black or near-white and destroyed the texture detail entirely, so the factor was reduced.

**Ordering and value range.** The augmentation layers operate on raw  $[0, 255]$  pixels and return values in the same range, so any model-specific normalisation (a `Rescaling` layer for the custom CNN, or the backbone’s own `preprocess_input` for the transfer models) is applied *after* augmentation. Placing rescaling first is a subtle but real bug: the brightness layer is configured for a  $[0, 255]$  value range, and feeding it  $[0, 1]$  inputs saturates the image to black. Because augmentation lives inside the model graph rather than in the data pipeline, this ordering is fixed at construction time and reduces the risk of an inference-time ordering mismatch.

**Augmentation is training-only.** The Keras preprocessing layers are inactive outside training, and the tuning and held-out datasets are never passed through the augmentation pipeline. All reported metrics therefore reflect performance on clean, untouched images.

**Status as a controlled variable.** The policy is fixed and shared rather than tuned per model. Individual architectures might well prefer slightly different settings, but allowing each member to tune augmentation independently would confound the comparison: a difference in final accuracy could then reflect a better augmentation search rather than a better architecture. Holding

augmentation constant — along with the data, split, external input contract, and evaluation procedure — reduces these sources of confounding and supports the architecture-focused comparison in Section 6.2.

## 4 Deep Learning Approaches

### 4.1 Custom CNN (from scratch)

The custom CNN is the only approach in this study that uses no pretrained weights. It contains 26,512,212 parameters in total: 26,511,444 trainable parameters and 768 non-trainable BatchNorm moving-mean and moving-variance state parameters. Its role is therefore twofold: it is a competitive architecture in its own right, and it is the experimental control that makes the contribution of ImageNet pretraining measurable rather than assumed — a comparison the other four approaches cannot provide on their own.

#### 4.1.1 Source Architecture

Rather than assemble an arbitrary stack of layers, we reproduce the *Pyramidal Net* architecture proposed by Abdelmonem et al. [1], which was designed specifically for remote-sensing scene classification and reports 94.28% accuracy on a ten-class NWPU-RESISC45 subset and 93% on UC Merced. NWPU-RESISC45 contains 700 images per class, but the source experiment allocates them 70/15/15 to training, validation, and test, so its nominal training allocation is 490 images per class rather than our provided 700-image training pool. The shared  $256 \times 256$  resolution makes the architecture relevant, while the different data budget prevents a like-for-like performance comparison.

The name describes the design principle: spatial dimensions contract while feature abstraction increases, so the representation narrows toward the classifier like a pyramid. Table 6 gives the structure as implemented.

**Table 6:** Custom CNN (Pyramidal Net) architecture, following [1].

| Stage                     | Configuration                                              | Output                             |
|---------------------------|------------------------------------------------------------|------------------------------------|
| Input                     | $256 \times 256 \times 3$ , raw $[0, 255]$ RGB             | $256^2 \times 3$                   |
| Augmentation              | shared team policy, training only                          | $256^2 \times 3$                   |
| Rescaling                 | $\times 1/255$                                             | $256^2 \times 3$                   |
| Conv 1                    | 256 filters, $3 \times 3$ , ReLU, valid                    | $254^2 \times 256$                 |
| MaxPool 1 + BatchNorm     | $2 \times 2$ , valid                                       | $127^2 \times 256$                 |
| Conv 2                    | 128 filters, $3 \times 3$ , ReLU, valid                    | $125^2 \times 128$                 |
| MaxPool 2 + BatchNorm     | $2 \times 2$ , valid                                       | $62^2 \times 128$                  |
| Conv 3                    | 128 filters, $3 \times 3$ , ReLU, valid                    | $60^2 \times 128$                  |
| Conv 4                    | 128 filters, $3 \times 3$ , ReLU, valid                    | $58^2 \times 128$                  |
| Conv 5                    | 64 filters, $3 \times 3$ , ReLU, valid                     | $56^2 \times 64$                   |
| MaxPool 3                 | $2 \times 2$ , valid                                       | $28^2 \times 64$                   |
| Flatten                   | —                                                          | 50,176                             |
| Dense block               | $512 \rightarrow 256 \rightarrow 64 \rightarrow 16$ , ReLU | 16                                 |
| Output                    | Dense, softmax                                             | 4                                  |
| Parameters                |                                                            | 26,512,212 total                   |
| Trainable / non-trainable |                                                            | 26,511,444 / 768 (BatchNorm state) |

The flattened vector of 50,176 features matches the source paper exactly, confirming a faithful reproduction of the convolutional stack. All five convolutional layers use  $3 \times 3$  kernels, unit stride, valid padding, and ReLU activation, and batch normalisation follows the first two pooling stages, all as specified in [1].

#### 4.1.2 Training Configuration

The optimisation settings follow the source paper’s Table 4. The learning-rate schedule in particular is specific to this architecture and is supplied through the shared training entrypoint’s per-model callback hook, so it applies to this model alone and does not alter any teammate’s run.

**Table 7:** Custom CNN training configuration, after [1] Table 4.

| Hyperparameter            | Source paper                    | This work                       |
|---------------------------|---------------------------------|---------------------------------|
| Optimiser                 | Adam                            | Adam                            |
| Initial learning rate     | 0.00025                         | 0.00025                         |
| LR scheduler              | ReduceLROnPlateau               | ReduceLROnPlateau               |
| LR reduction factor       | 0.25                            | 0.25                            |
| Scheduler patience        | 4 epochs                        | 4 epochs                        |
| Minimum learning rate     | $1 \times 10^{-6}$              | $1 \times 10^{-6}$              |
| Weight decay              | L2 norm term                    | L2 ( $10^{-4}$ ) on conv layers |
| Dropout                   | none                            | none                            |
| Kernel / stride / padding | $3 \times 3 / 1 / \text{valid}$ | identical                       |
| Activation                | ReLU                            | ReLU                            |
| Loss                      | sparse categorical CE           | categorical CE (one-hot)        |
| Epochs                    | 30                              | 32                              |
| Batch size                | 8                               | 16 (team standard)              |
| Random seed               | 32                              | 42 (team standard)              |
| Split                     | 70/15/15                        | 74/13/13 (fixed test set)       |
| Normalisation             | resize only                     | resize + rescale to $[0, 1]$    |

Four deviations are worth justifying explicitly. First, the loss is categorical cross-entropy over one-hot labels rather than the sparse variant; these are mathematically identical objectives differing only in label encoding, and the one-hot form is what the shared data pipeline emits for all five models. Second, batch size, seed, and split are dictated by the team-wide controlled variables described in Section 3.1, without which the five approaches would not be comparable. Third, the source applies no pixel normalisation, keeping inputs in  $[0, 255]$ ; we insert a rescaling layer after augmentation because a network trained from random initialisation optimises more stably on normalised inputs, and the batch-normalisation layers make the change largely self-correcting in any case. Fourth, the paper specifies weight decay as “an L2 norm term” without a coefficient; we apply  $10^{-4}$  to the convolutional layers, leaving the dense head unregularised.

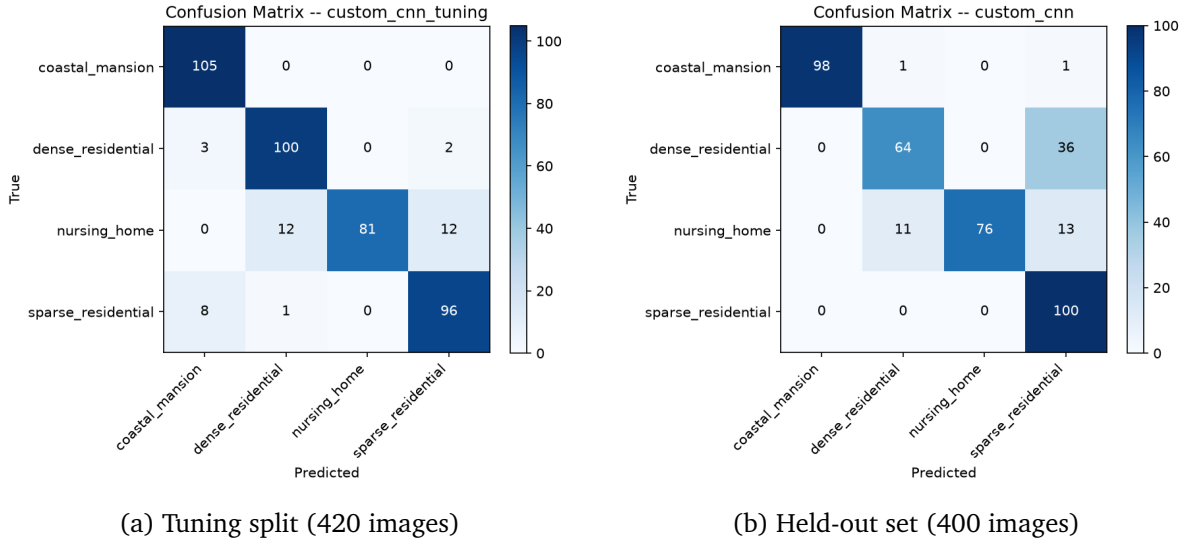
#### 4.1.3 Results

Trained under the shared protocol at batch size 16 for 32 epochs, the model selected its best checkpoint at epoch 24 and achieved **84.50% accuracy** and **0.8452 macro- $F_1$**  on the held-out set — the lowest of our five approaches. The per-class breakdown localises the shortfall precisely:

| Class              | $F_1$        |
|--------------------|--------------|
| coastal_mansion    | 0.990        |
| nursing_home       | 0.864        |
| sparse_residential | 0.800        |
| dense_residential  | <b>0.727</b> |

Performance is essentially saturated on coastal\_mansion, whose shoreline provides an unambiguous global cue, and collapses on dense\_residential. The confusion matrix in Figure 2

shows that this is not diffuse error but a single dominant confusion: `dense_residential` is misclassified as `sparse_residential` in a large fraction of cases, while `sparse_residential` itself is recovered almost perfectly. The error is therefore *asymmetric* — the model under-predicts density rather than confusing the two classes symmetrically.



**Figure 2:** Custom CNN confusion matrices. (a) The tuning split, on which the checkpoint was selected. (b) The sealed held-out set. Errors concentrate almost entirely in the `dense_residential`  $\rightarrow$  `sparse_residential` direction, and that confusion is markedly worse on the held-out data — an observed tuning-to-held-out gap consistent with selection optimism.

This is exactly the failure mode the exploratory analysis predicted: the two classes share an identical visual vocabulary and are separated only by a continuous density threshold, so the distinction requires fine-grained texture discrimination rather than scene-level cues.

**The observed selection gap is class-specific.** Comparing the two panels of Figure 2 quantifies the tuning-to-held-out difference after selecting the checkpoint on tuning accuracy, and shows that it is not spread evenly across classes (Table 8).

**Table 8:** Per-class  $F_1$  on the tuning split versus the held-out set.

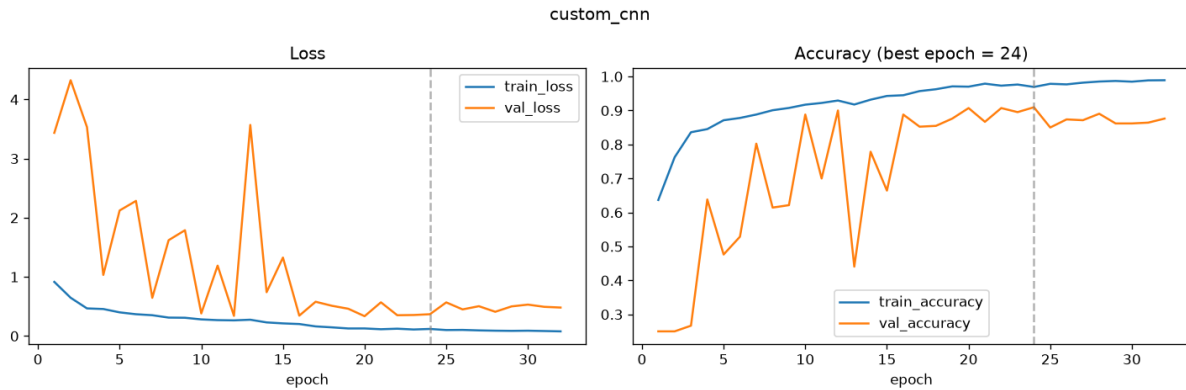
| Class              | Tuning | Held-out | $\Delta$ |
|--------------------|--------|----------|----------|
| coastal_mansion    | 0.950  | 0.990    | +0.040   |
| nursing_home       | 0.871  | 0.864    | −0.007   |
| sparse_residential | 0.893  | 0.800    | −0.093   |
| dense_residential  | 0.917  | 0.727    | −0.190   |
| Macro- $F_1$       | 0.9079 | 0.8452   | −0.0627  |

Three of the four classes transfer with little or no loss, and `coastal_mansion` in fact improves. Essentially the entire 6.3-point macro- $F_1$  drop is attributable to `dense_residential`, which falls by 19 points. On the tuning split the model appears to have largely solved that class at  $F_1 =$

0.917; on unseen data it has not, at  $F_1 = 0.727$ . The selected checkpoint is thus the epoch at which the model best fitted the particular density boundary present in those 420 tuning images, and that boundary does not generalise.

This is a concrete illustration of why Section 3.1 insists on a sealed held-out set. Had the tuning metrics been reported as final — a natural mistake, since they are computed on data the model never received gradients from — we would have overstated this model’s performance by more than six points and, more seriously, drawn the wrong conclusion about which class remains unsolved.

The training curves (Figure 3) reveal two distinct phases. Through roughly the first fifteen epochs the model exhibits pronounced validation instability, with tuning accuracy oscillating between 0.25 and 0.90 and validation loss spiking above 4.0, while training accuracy rises smoothly throughout. One plausible explanation is slow convergence of the batch-normalisation moving averages, which are estimated from batch statistics during training but applied as running averages at validation time. After the first learning-rate reduction at epoch 16, the oscillation collapses and validation loss flattens. This timing is consistent with the scheduler helping stabilise training, although no scheduler or BatchNorm ablation was run.



**Figure 3:** Custom CNN training curves. The dashed line marks epoch 24, the checkpoint selected on tuning accuracy. Early validation instability may reflect batch-normalisation statistics converging on a small tuning split; it subsides after the learning-rate scheduler first fires at epoch 16. The persistent gap between the training and validation curves after epoch 20 indicates residual overfitting.

The second phase exposes the model’s central limitation. Training accuracy continues to climb toward 0.99 while validation accuracy plateaus near 0.87 and validation loss ceases to improve, leaving a gap of roughly twelve points. The L2 term reduces this overfitting but does not eliminate it, which is unsurprising given that 25.7M of the 26.5M parameters reside in the first dense layer alone — 97% of the model’s capacity is concentrated in a single layer fitted to 2,378 images.

#### 4.1.4 Discussion

Our result does not reproduce the ordering reported in [1], where the custom architecture is presented as competitive with or superior to pretrained networks. On our partition it is decisively outperformed by all four transfer-learning approaches, including MobileNetV2, which

reaches 0.9576 macro- $F_1$  with 2.26M parameters — roughly one twelfth of the custom model’s capacity.

One plausible explanation for the discrepancy is the granularity of the classification task rather than a defect in the architecture. The ten NWPU-RESISC45 categories used in the source study — harbour, chaparral, tennis court, industrial area, parking lot, forest, beach, overpass, airplane, and baseball diamond — are semantically and visually disjoint, and are separable on coarse cues such as dominant colour, the presence of water, or large geometric structures. Features sufficient for those distinctions are learnable from 700 images per class. Our four categories are all residential or institutional building scenes distinguished by building density and spatial arrangement, and the discriminative features are correspondingly subtler. Our results are consistent with ImageNet pretraining supplying useful mid-level texture and structure features that the from-scratch model did not learn as effectively from 2,378 images; the experiment does not isolate that mechanism directly.

It is also worth noting that the source paper’s own comparison is more equivocal than its abstract suggests: on NWPU-RESISC45 (its Table 7) VGG16 attains 0.949 and Xception 0.948 test accuracy against the proposed model’s 0.9428, so the architecture’s advantage there rests on its efficiency and its markedly stronger performance on the smaller UC Merced dataset rather than on raw accuracy. Read in that light, our finding is less a contradiction of [1] than a delineation of where its conclusions hold: a well-designed from-scratch CNN is competitive on coarse-grained scene classification, but pretraining remains decisive when categories are fine-grained and data is limited. Establishing that boundary is precisely the contribution this baseline makes to our comparison, and it is developed further in Section 7.

## 4.2 ResNet-50 (transfer learning)

ResNet’s key idea is the residual (skip) connection: each block learns an adjustment to its input rather than a full transformation, allowing gradients to flow through many layers and enabling genuinely deep networks to train effectively [5]. ResNet-50, pretrained on ImageNet, is one of the most widely used backbones in computer vision, with well-understood transfer behaviour that makes it a natural strong default among our five approaches.

We use the backbone as a frozen feature extractor and train only a small new classification head. The whole pipeline is packaged inside a single Keras model (Table 9): the saved checkpoint accepts raw  $[0, 255]$  RGB images and handles augmentation and preprocessing itself.

**Table 9:** ResNet-50 model configuration.

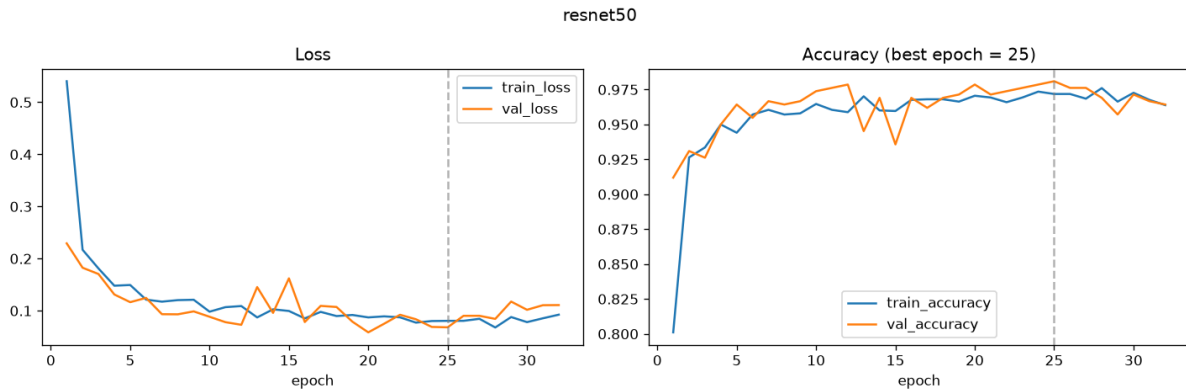
| Stage         | Details                                                               |
|---------------|-----------------------------------------------------------------------|
| Input         | $256 \times 256 \times 3$ , raw $[0, 255]$ RGB                        |
| Augmentation  | shared team policy, active during training only                       |
| Preprocessing | ResNet <code>preprocess_input</code> (ImageNet channel normalisation) |
| Backbone      | ResNet-50, ImageNet weights, frozen, global average pooling           |
| Head          | dropout 0.3 $\rightarrow$ dense softmax (4 units)                     |
| Parameters    | 23,595,908 total; 8,196 trainable                                     |
| Optimisation  | Adam, learning rate $10^{-3}$ , categorical cross-entropy             |

Three design decisions are worth spelling out. First, the backbone is frozen rather than fine-

tuned: with 2,378 training images against 23.6M parameters, full fine-tuning risks overfitting, and freezing matches what the other transfer models do, keeping the comparison clean. Only the 8,196 parameter head is fitted to our data. Second, augmentation and preprocessing live inside the model graph rather than in the data pipeline, reducing the risk of a training–serving normalisation mismatch: the same graph-level preprocessing is used during training and inference. Third, the two head hyper-parameters (dropout rate and learning rate) are exposed as controlled experiment factors; the chosen values (0.3 and  $10^{-3}$ ) were selected on the internal tuning split and never against the held-out set.

#### 4.2.1 Results

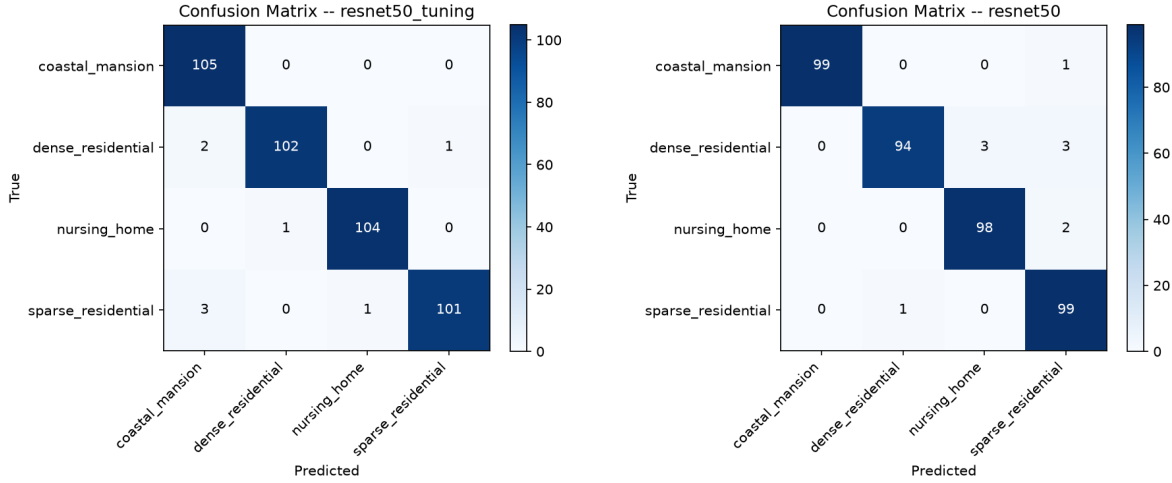
Training used the shared protocol — batch size 16, up to 32 epochs, early stopping with patience 20 — and selected the best checkpoint at epoch 25. On the held-out set the model achieves **97.5% accuracy** and **0.9750 macro- $F_1$** , the second strongest of our five approaches. Per-class  $F_1$  ranges narrowly from 0.964 (dense\_residential) to 0.995 (coastal\_mansion), so no class is left behind. The full leaderboard is given in Section 6.2.



**Figure 4:** ResNet-50 training curves. The dashed line marks epoch 25, the selected checkpoint. Training and validation curves track each other closely throughout, and validation accuracy frequently exceeds training accuracy because augmentation is active only on the training stream.

The curves in Figure 4 differ markedly from those of the from-scratch model. Validation accuracy exceeds 0.91 after a single epoch and settles above 0.95 within five, reflecting that the frozen ImageNet features are already discriminative for this task and only the 8,196-parameter head has to be fitted. There is no widening gap between the training and validation curves; indeed validation accuracy frequently sits *above* training accuracy, which is expected here because augmentation is applied to the training stream only, so the model is scored on clean images at validation time but on flipped, rotated, and jittered ones during training. With so few trainable parameters the model has almost no capacity to memorise the training set, and the residual oscillation around epochs 13–16 reflects the small size of the 420-image tuning split rather than instability in the model.

**The observed tuning-to-held-out gap is small.** Repeating the tuning-versus-held-out comparison made for the custom CNN (Table 10) produces a very different picture. Every per-class



(a) Tuning split (420 images)

(b) Held-out set (400 images)

**Figure 5:** ResNet-50 confusion matrices. Unlike the from-scratch model, the two panels are closely similar: the frozen backbone leaves little room for the checkpoint to overfit the tuning split.

difference is within  $\pm 0.02 F_1$ , and macro- $F_1$  falls by only 0.006 — roughly one tenth of the 0.063 drop seen in the from-scratch model.

**Table 10:** ResNet-50 per-class  $F_1$  on the tuning split versus the held-out set.

| Class              | Tuning | Held-out | $\Delta$ |
|--------------------|--------|----------|----------|
| coastal_mansion    | 0.977  | 0.995    | +0.018   |
| sparse_residential | 0.976  | 0.966    | -0.010   |
| nursing_home       | 0.990  | 0.975    | -0.015   |
| dense_residential  | 0.981  | 0.964    | -0.017   |
| Macro- $F_1$       | 0.9810 | 0.9750   | -0.0059  |

The contrast is consistent with trainable capacity contributing to selection overfit, but one run cannot isolate capacity from architecture or sampling variation. ResNet-50 adapts 8,196 parameters and shows a 0.6-point observed drop; the custom CNN adapts 26,511,444 and shows a 6.3-point drop concentrated almost entirely in one class. Freezing the backbone therefore coincides here with both stronger accuracy and a more stable *validation signal*, without establishing that it alone caused either result. The relationship is examined across all five models in Section 7.

### 4.3 EfficientNet-B0 (transfer learning)

EfficientNet was designed around a different question from ResNet: once a competent convolutional network exists, how should additional computation be distributed between depth, width, and input resolution? Tan and Le [12] show that scaling only one dimension produces an unbalanced network and propose a compound coefficient that increases all three together.

EfficientNet-B0 is the neural-architecture-search baseline from which the larger B1–B7 family is scaled. Its main operator is the mobile inverted bottleneck (MBConv), inherited from MobileNetV2, augmented with squeeze-and-excitation channel attention.

That design makes B0 a useful contrast to both ResNet-50 and MobileNetV2 in this study. Against ResNet-50 it asks whether a much smaller, explicitly efficiency-designed backbone can preserve fine-grained accuracy. Against MobileNetV2 it asks whether squeeze-and-excitation and EfficientNet’s searched stage allocation buy accuracy beyond the common MBConv foundation.

### 4.3.1 Architecture and Adaptation

The Keras ImageNet checkpoint is used without its original 1,000-class head. Global average pooling produces a 1,280-dimensional representation, followed by the same dropout-plus-softmax head used for MobileNetV2. The complete configuration is shown in Table 11.

**Table 11:** EfficientNet-B0 model configuration.

| Stage         | Details                                                           |
|---------------|-------------------------------------------------------------------|
| Input         | $256 \times 256 \times 3$ , raw $[0, 255]$ RGB                    |
| Augmentation  | shared team policy, active during training only                   |
| Preprocessing | Keras EfficientNet input handling inside the saved graph          |
| Backbone      | EfficientNet-B0, ImageNet weights, frozen, global average pooling |
| Head          | dropout 0.3 $\rightarrow$ dense softmax (4 units)                 |
| Parameters    | 4,054,695 total; 5,124 trainable                                  |
| Optimisation  | Adam, learning rate $10^{-3}$ , categorical cross-entropy         |

Only 5,124 parameters — 0.13% of the network — are updated. This is a deliberate transfer-learning decision rather than a computational shortcut. The training subset has 2,378 images, too few to estimate four million backbone parameters reliably, while the ImageNet filters already encode edges, textures, repeated structures, and spatial compositions relevant to aerial scenes. Freezing also keeps the evaluation aligned with the other pretrained CNNs: their representation quality is compared under a common four-class head rather than confounded by different fine-tuning depths.

Our implementation makes three project-specific modifications. First, it accepts the shared native  $256 \times 256$  input rather than the paper’s  $224 \times 224$  baseline resolution; global pooling permits this without changing the classifier dimensions. Second, the shared aerial augmentation is embedded before EfficientNet’s preprocessing so the saved checkpoint accepts raw images in both training and deployment. Third, the original 1,000-class classifier is replaced by a four-class head whose dropout and learning rate were exposed through the shared experiment interface. The selected values were chosen from the internal tuning split before the held-out evaluation.

### 4.3.2 Position against Published Architectures

The ImageNet figures in Table 12 are included to position the architecture, not to compare dataset difficulty with our four-class PatternNet partition. In the original paper, B0 matches or slightly exceeds substantially larger models at the same ImageNet accuracy level [12].

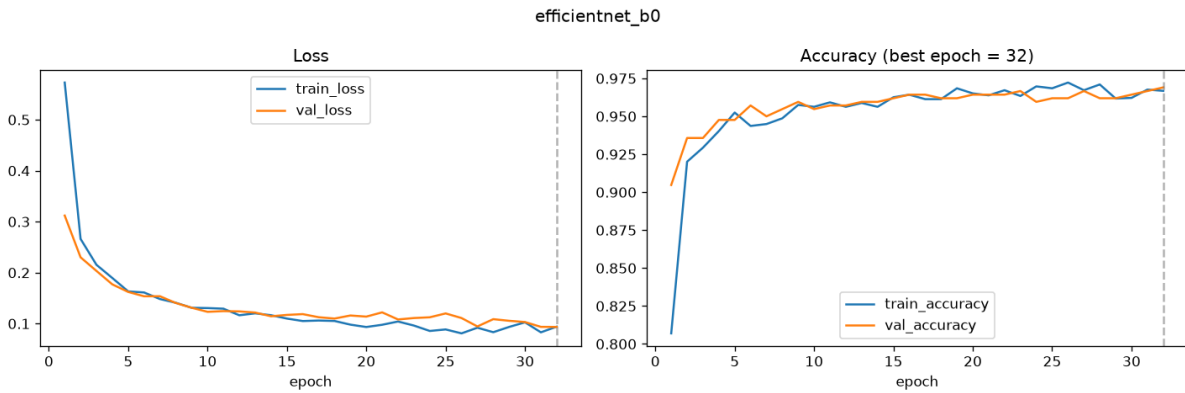
**Table 12:** Published ImageNet positioning of EfficientNet-B0 against architectures not otherwise implemented in this project, reproduced from [12].

| Architecture    | Top-1 | Parameters | FLOPs |
|-----------------|-------|------------|-------|
| EfficientNet-B0 | 76.3% | 5.3M       | 0.39B |
| ResNet-50       | 76.0% | 26M        | 4.1B  |
| DenseNet-169    | 76.2% | 14M        | 3.5B  |

The comparison explains why B0 was a sensible candidate before seeing our results: its published advantage is not a claim of universally higher accuracy, but a claim of better accuracy per unit of model capacity and computation. Our study tests whether that efficiency transfers to a fine-grained remote-sensing task under a frozen-backbone protocol. The answer is positive within our five-model scope: B0 uses about one sixth of ResNet-50’s parameters yet finishes one image ahead on the 400-image held-out set. That one-image difference is not statistically sufficient to establish universal superiority, but the parameter reduction remains material regardless of that ordering.

#### 4.3.3 Results

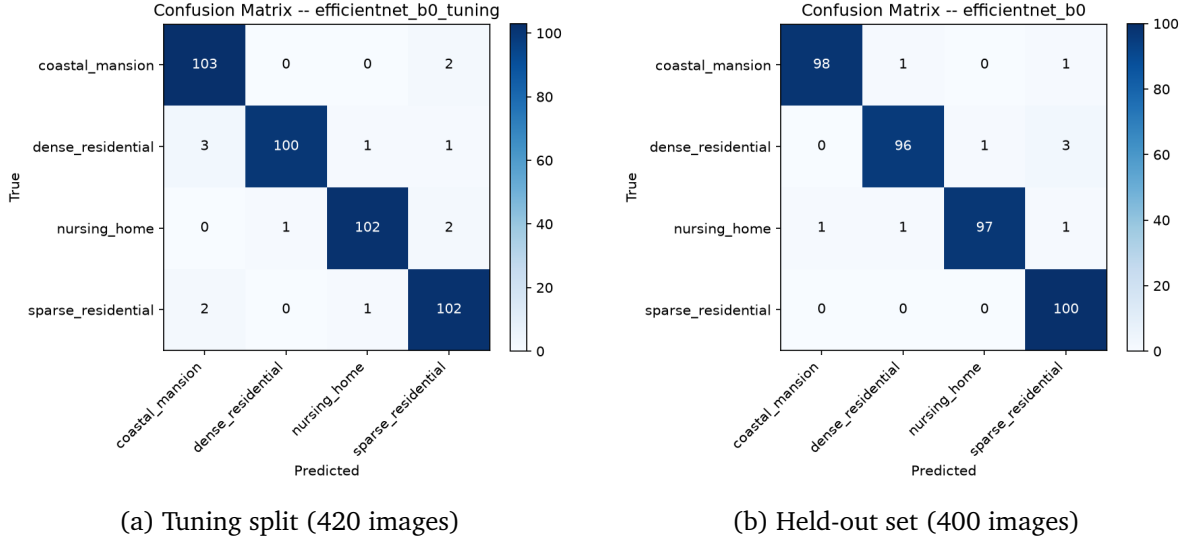
Training used batch size 16, a 32-epoch budget, and early-stopping patience 20. The highest tuning accuracy occurred at epoch 32, so the final checkpoint lies at the boundary of the budget rather than at an early plateau. Figure 6 shows rapid transfer in the first epochs followed by smaller improvements throughout training. Validation accuracy rises from 0.905 after epoch 1 to 0.969 at epoch 32 while validation loss continues to decline, providing no evidence of late-stage divergence.



**Figure 6:** EfficientNet-B0 training curves. The dashed line marks epoch 32, the selected checkpoint. The absence of a late validation-loss increase supports the frozen-backbone choice, although the boundary optimum suggests that a longer pre-registered epoch budget is worth testing in future work.

On the held-out set EfficientNet-B0 achieves **97.75% accuracy** and **0.9775 macro- $F_1$** , the highest observed values in this study. It misclassifies 9 of 400 images. All four class  $F_1$  values are at least 0.970: `coastal_mansion` reaches 0.985, `nursing_home` 0.980, `sparse_residential`

0.976, and `dense_residential` 0.970. The largest cell off the diagonal is three `dense_residential` images predicted as `sparse_residential`, matching the density boundary identified during EDA.



**Figure 7:** EfficientNet-B0 confusion matrices. Errors remain sparse across both splits; the held-out set contains 9 errors versus 13 on the tuning split.

**Table 13:** EfficientNet-B0 per-class  $F_1$  on tuning and held-out data.

| Class              | Tuning | Held-out | $\Delta$ |
|--------------------|--------|----------|----------|
| coastal_mansion    | 0.967  | 0.985    | +0.018   |
| dense_residential  | 0.971  | 0.970    | -0.001   |
| nursing_home       | 0.976  | 0.980    | +0.004   |
| sparse_residential | 0.962  | 0.976    | +0.013   |
| Macro- $F_1$       | 0.9691 | 0.9775   | +0.0084  |

The positive held-out delta should not be interpreted as evidence that the held-out set is easier in general: it is only a 400-image sample, and the absolute difference is approximately three correctly classified images. Instead, the close tuning/held-out agreement shows that the 5,124-parameter head did not specialise strongly to the tuning subset. EfficientNet-B0’s main weakness is therefore not observed overfitting but unfinished convergence: the best epoch is the last available epoch. A longer schedule or partial fine-tuning may improve it, but those are future experiments and cannot be introduced after inspecting held-out errors.

#### 4.4 MobileNetV2 (transfer learning)

MobileNetV2 is the explicitly resource-constrained member of our comparison. Its basic unit is an inverted residual with a linear bottleneck [9]. A conventional residual block compresses a wide representation and then expands it again; MobileNetV2 reverses that pattern. Each block expands a narrow input with a  $1 \times 1$  convolution, filters the expanded channels independently

with a depthwise  $3 \times 3$  convolution, and projects them linearly back to a narrow output. Skip connections join the thin bottlenecks when shape and stride permit.

Two details are important. Depthwise separable convolution avoids paying for a full spatial kernel across every input-output channel pair, while the linear final projection avoids destroying information with ReLU in the low-dimensional bottleneck. The source paper reports that a  $3 \times 3$  depthwise-separable operation is roughly eight to nine times cheaper than a standard convolution at comparable channel dimensions [9]. This makes MobileNetV2 a principled efficiency baseline rather than simply a smaller arbitrary CNN.

#### 4.4.1 Architecture and Adaptation

**Table 14:** MobileNetV2 model configuration.

| Stage         | Details                                                                 |
|---------------|-------------------------------------------------------------------------|
| Input         | $256 \times 256 \times 3$ , raw $[0, 255]$ RGB                          |
| Augmentation  | shared team policy, active during training only                         |
| Preprocessing | MobileNetV2 <code>preprocess_input</code> , mapping to $[-1, 1]$        |
| Backbone      | MobileNetV2 width 1.0, ImageNet weights, frozen, global average pooling |
| Head          | dropout 0.3 $\rightarrow$ dense softmax (4 units)                       |
| Parameters    | 2,263,108 total; 5,124 trainable                                        |
| Optimisation  | Adam, learning rate $10^{-3}$ , categorical cross-entropy               |

The implementation uses the standard width multiplier 1.0 backbone without its ImageNet classifier. As with EfficientNet-B0, only the 5,124-parameter head is trained. The shared augmentation is followed by MobileNetV2 preprocessing inside the model graph, so training, held-out evaluation, and deployment all consume the same raw  $[0, 255]$  input contract. The project-specific changes are the native  $256 \times 256$  input, four-class head, shared augmentation, frozen feature-extraction policy, and common experiment metadata. These changes retain the source architecture while adapting its input and output contracts to the assigned aerial partition.

#### 4.4.2 Position against Published Mobile Models

Table 15 reproduces the source paper’s ImageNet comparison with architectures outside our project [9]. These numbers compare resource design points on ImageNet and must not be read as baselines for our PatternNet accuracy.

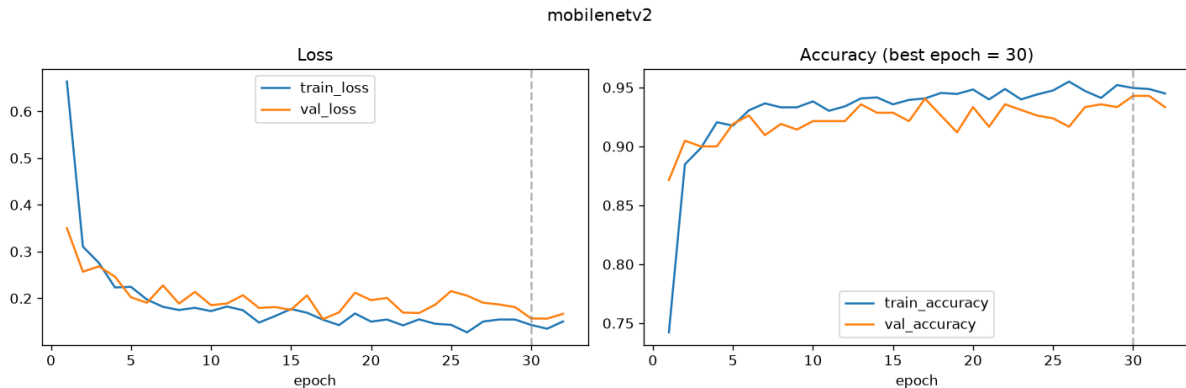
**Table 15:** Published ImageNet positioning of MobileNetV2 against mobile architectures not implemented in this project, from [9]. CPU latency is measured by the authors on one Google Pixel 1 core.

| Architecture      | Top-1 | Parameters | MAdds | CPU latency |
|-------------------|-------|------------|-------|-------------|
| MobileNetV1       | 70.6% | 4.2M       | 575M  | 113 ms      |
| ShuffleNet (1.5)  | 71.5% | 3.4M       | 292M  | —           |
| NASNet-A          | 74.0% | 5.3M       | 564M  | 183 ms      |
| MobileNetV2       | 72.0% | 3.4M       | 300M  | 75 ms       |
| MobileNetV2 (1.4) | 74.7% | 6.9M       | 585M  | 143 ms      |

The paper’s width-1.0 MobileNetV2 improves on MobileNetV1 while reducing parameters, operations, and measured latency; NASNet-A is more accurate but larger and slower on the cited device. Our study does not reproduce those hardware measurements, so we make no latency claim for our container. What we can verify directly is model capacity: with its ImageNet head removed, our MobileNetV2 has 2.26M parameters, the smallest of the five approaches. It therefore establishes the low-capacity end of our own accuracy-versus-size comparison.

#### 4.4.3 Results

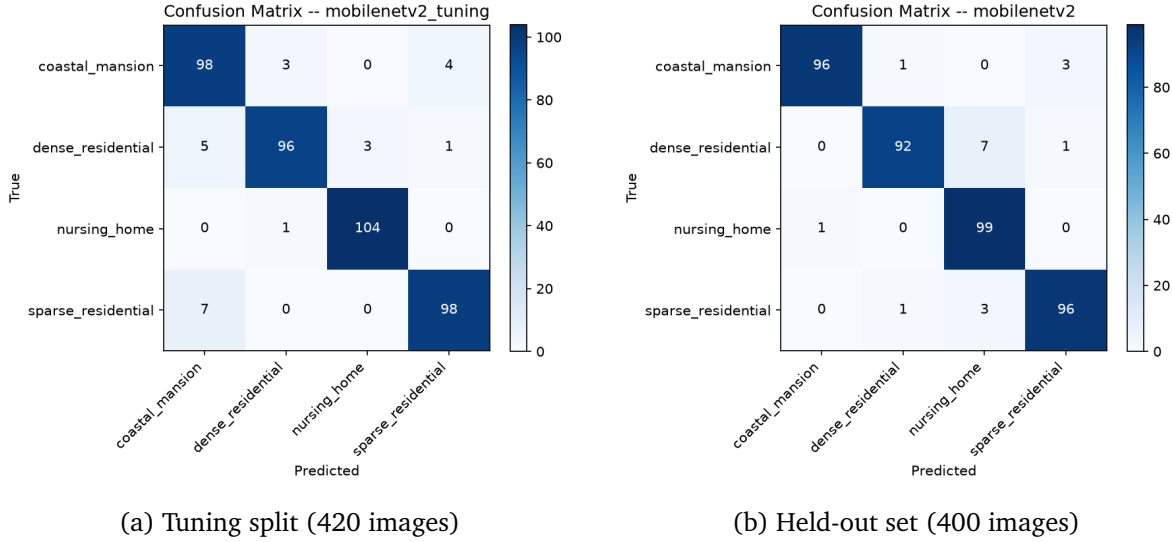
MobileNetV2 trains for all 32 requested epochs and selects epoch 30. Its first tuning accuracy is already 0.871, rises past 0.90 at epoch 2, and reaches 0.943 at the selected checkpoint. Figure 8 shows a noisier validation trajectory than EfficientNet-B0. This is consistent with scoring a small 420-image tuning set: a change of one prediction moves accuracy by about 0.24 percentage points, so visible oscillation need not imply unstable optimisation.



**Figure 8:** MobileNetV2 training curves. The dashed line marks epoch 30, the selected checkpoint. Training continues to improve gradually while tuning accuracy oscillates within a narrow band near the end of the run.

On held-out data MobileNetV2 reaches **95.75% accuracy** and **0.9576 macro- $F_1$** , corresponding to 17 errors. This is fourth in raw accuracy but first in compactness. The result is therefore not a failed attempt to maximise the leaderboard; it quantifies the cost of choosing the smallest deployable representation in the study. Relative to EfficientNet-B0, MobileNetV2 gives up 2.0 accuracy points while using 44% fewer parameters.

The held-out per-class pattern is uneven despite the balanced dataset. `coastal_mansion` is strongest at  $F_1 = 0.975$ . `dense_residential` and `nursing_home` both fall near 0.948: seven dense-residential images are predicted as nursing home, while the latter retains 0.99 recall at the cost of 0.908 precision. The model has therefore learned a permissive nursing-home decision region that captures nearly every true nursing home but also absorbs dense residential complexes with similar large-building layouts.



**Figure 9:** MobileNetV2 confusion matrices. The held-out matrix shifts from coastal/sparse errors toward dense-residential images predicted as nursing home, showing that the aggregate score hides a changed error mechanism.

**Table 16:** MobileNetV2 per-class  $F_1$  on tuning and held-out data.

| Class              | Tuning | Held-out | $\Delta$ |
|--------------------|--------|----------|----------|
| coastal_mansion    | 0.912  | 0.975    | +0.063   |
| dense_residential  | 0.937  | 0.948    | +0.012   |
| nursing_home       | 0.981  | 0.947    | -0.034   |
| sparse_residential | 0.942  | 0.960    | +0.018   |
| Macro- $F_1$       | 0.9429 | 0.9576   | +0.0147  |

The macro score rises on held-out data, but the class-level reversal is the more informative result: the tuning split understates coastal performance and overstates nursing-home precision. MobileNetV2’s principal strength is the verified parameter reduction with only a modest accuracy cost. Its weakness is less separation between dense residential and institution-scale buildings, suggesting that the compact representation discards some of the global layout detail retained by EfficientNet-B0 and ResNet-50. Partial fine-tuning of the last inverted-residual stages is the natural next experiment, but it must be selected on internal tuning data rather than fitted in response to these held-out confusions.

## 4.5 Vision Transformer (ViT)

The Vision Transformer [4] applies a pure Transformer architecture to images: the input is split into fixed  $16 \times 16$  patches, each patch is linearly embedded and given a learned position embedding, and the resulting sequence is processed by a stack of self-attention layers. Unlike a CNN, whose convolutions can only relate nearby pixels layer by layer, self-attention lets every patch attend to every other patch from the first layer onward, capturing global layout cues

directly. This makes ViT a structurally different approach from our four convolutional models, which is exactly the contrast our comparison needs: aerial scene categories such as dense versus sparse residential are distinguished less by any single local texture than by the overall spatial arrangement of buildings, roads, and vegetation. Pretrained ViTs have already been shown to transfer effectively to remote-sensing scene classification at exactly this kind of scale [2].

We use the ViT-B/16 backbone pretrained on ImageNet as a frozen feature extractor and train only a small classification head on the CLS token embedding. As with the other approaches, the whole pipeline is packaged inside a single Keras model (Table 17): the saved checkpoint accepts raw  $[0, 255]$  RGB images and applies augmentation and preprocessing itself.

**Table 17:** ViT-B/16 model configuration.

| Stage         | Details                                                                   |
|---------------|---------------------------------------------------------------------------|
| Input         | $256 \times 256 \times 3$ , raw $[0, 255]$ RGB                            |
| Augmentation  | shared team policy, active during training only                           |
| Resize        | $256 \times 256 \rightarrow 224 \times 224$ (bilinear), see below         |
| Preprocessing | scale to $[-1, 1]$ (ViT convention, not ImageNet channel stats)           |
| Backbone      | ViT-B/16, ImageNet weights, frozen; 12 layers, 768 hidden dim, 12 heads   |
| Head          | CLS token $\rightarrow$ dropout 0.3 $\rightarrow$ dense softmax (4 units) |
| Parameters    | 85,801,732 total; 3,076 trainable                                         |
| Optimisation  | Adam, learning rate $10^{-3}$ , categorical cross-entropy                 |

The model resizes its input from the shared  $256 \times 256$  to  $224 \times 224$  immediately before the backbone. This is not optional: ViT’s learned position embeddings form a fixed table with one entry per patch position ( $14 \times 14$  patches + CLS = 197 tokens for a  $224 \times 224$  input), so a  $256 \times 256$  input would produce 257 tokens with no pretrained embedding to match. CNNs tolerate off-resolution input silently because convolution weights are shared across positions; ViT makes the mismatch explicit. Notably, the CNN backbones used by the other approaches were also pretrained at  $224 \times 224$ , so our ViT is arguably the only model evaluated at its native pretraining resolution. The resize is placed after the shared augmentation, so regularisation is applied identically to all five models. Second, the backbone is frozen: ViTs lack the inductive biases of convolutions and are known to need large datasets or heavy augmentation when trained or fine-tuned directly [4, 13], so with only 2,798 usable training images we fit just the 3,076-parameter head — by far the smallest trainable footprint of our five approaches, which limits its capacity to overfit the tuning split. Third, preprocessing scales pixels to  $[-1, 1]$  rather than applying ImageNet channel normalisation, matching how this checkpoint was pretrained.

**Choice of variant.** ViT-B/16 is the canonical member of the family — the configuration the original paper itself headline-reports [4] — and each alternative variant was excluded on grounds specific to this task. Patch-32 variants retain a quarter of the tokens ( $7 \times 7$  vs.  $14 \times 14$  at 224 pixels), discarding exactly the spatial resolution the fine-grained density classes depend on. The 384-pixel presets would *upsample* our native 256-pixel imagery for no information gain at three times the token count. Larger backbones (ViT-L, 307M parameters) could raise accuracy only marginally under a frozen linear probe [6] while worsening the cost-accuracy trade-off that Section 7 shows already disfavors ViT’s footprint. Partial fine-tuning of the upper transformer

blocks is the one variant with genuine potential to close the accuracy gap to the best CNNs, but it would break the frozen-backbone protocol that keeps the five-way comparison clean; we defer it to Section 12.

**Pretraining-source ablation.** The choice of pretrained weights was validated experimentally rather than assumed. The selected checkpoint was pretrained on ImageNet-21k and then finetuned on ImageNet-1k, following the AugReg release lineage [11]; we compared it against the raw ImageNet-21k weights (never finetuned to the 1k label space) under an otherwise identical protocol — same seed, split, augmentation, and training flags. Our hypothesis was that the more generic 21k features would transfer better to aerial imagery, which is a clear domain shift from natural photos. To keep the held-out set untouched by model selection, the comparison used the internal tuning split only (Table 18).

**Table 18:** ViT pretraining-source ablation on the internal tuning split.

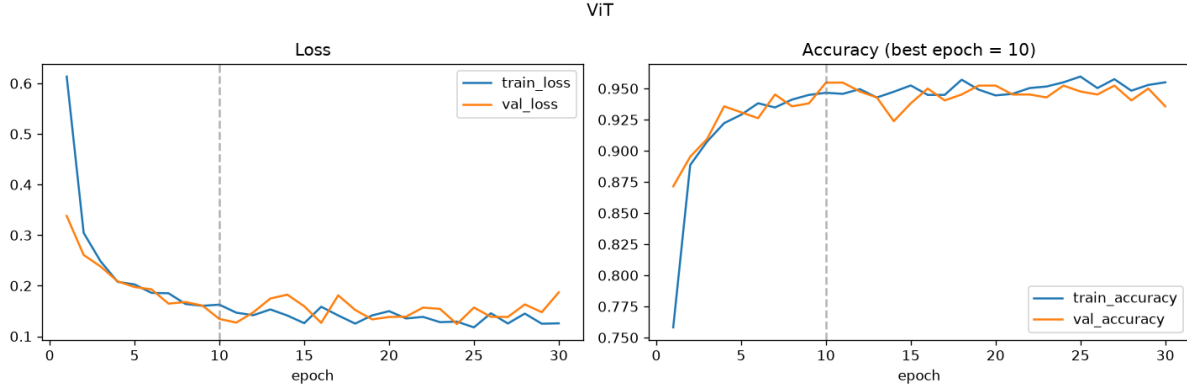
| Variant                          | Tuning accuracy | Tuning macro- $F_1$ | Best epoch |
|----------------------------------|-----------------|---------------------|------------|
| ImageNet-1k finetuned (selected) | 0.9548          | 0.9549              | 10         |
| ImageNet-21k raw                 | 0.9405          | 0.9408              | 31         |

The hypothesis was cleanly falsified: the 1k-finetuned checkpoint scored 1.4 points higher macro- $F_1$  and converged three times faster (best epoch 10 vs. 31, with the 21k variant still improving when the 32-epoch budget ended). The explanation lies in how the features are used. Supervised finetuning on the 1k label space shapes the CLS embedding to be linearly separable — precisely what a frozen-backbone setup with a softmax head requires, consistent with the strong dependence of penultimate-feature transfer on the pretraining objective observed by Kornblith et al. [6]. The raw 21k features are not poorer, but more entangled: a 3,076-parameter head can only draw four hyperplanes through the 768-dimensional embedding space and cannot re-mix features, so it benefits from geometry that finetuning has already organised. The slow crawl of the 21k run is the visible signature of that harder optimisation problem. With full fine-tuning the outcome could well reverse; in the frozen regime, the linear-separability effect dominated the domain-shift effect. The 1k variant was therefore retained, and only it was evaluated on the held-out set.

#### 4.5.1 Results

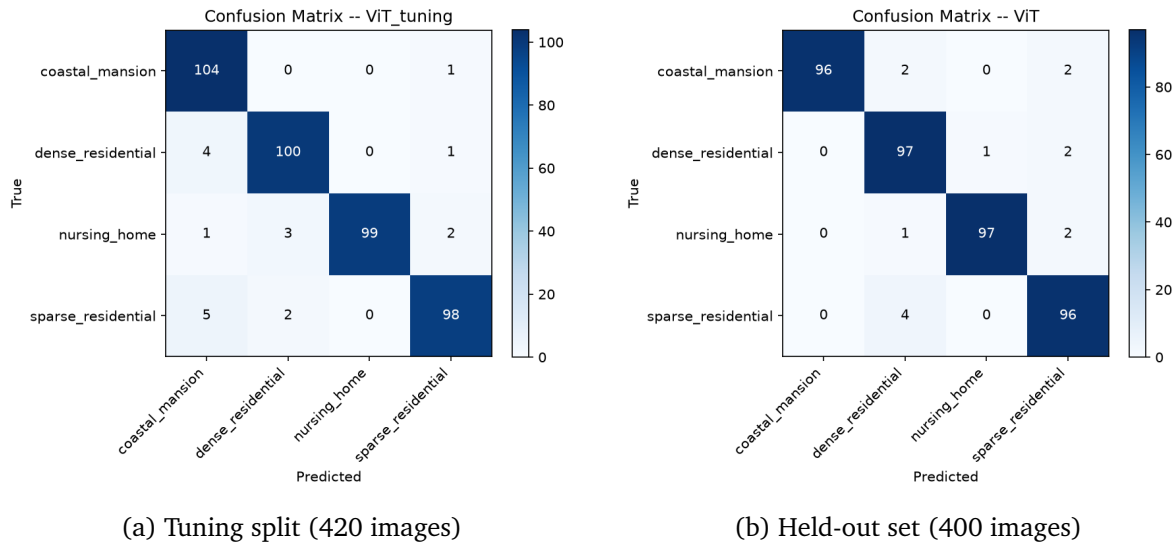
Training used the shared protocol — batch size 16, up to 32 epochs, early stopping with patience 20 — and selected the best checkpoint at epoch 10, the earliest of our five approaches; training stopped early at epoch 30 (best epoch plus patience). On the held-out set the model achieves **96.5% accuracy** and **0.9652 macro- $F_1$** , third of the five approaches and within 1.25 points of the strongest. Per-class  $F_1$  ranges from 0.951 (`sparse_residential` and `dense_residential`) to 0.980 (`nursing_home` and `coastal_mansion`, the latter with perfect precision — no image of any other class was ever predicted as a coastal mansion). The full leaderboard is given in Section 6.2.

The curves in Figure 10 show the fastest convergence of our five approaches, consistent with the ablation’s interpretation: the 1k-finetuned CLS embedding is already organised for linear clas-



**Figure 10:** ViT training curves. The dashed line marks epoch 10, the selected checkpoint. Validation accuracy exceeds 0.87 after a single epoch and plateaus within ten: the frozen ViT features are immediately usable and the 3,076-parameter head has little left to learn thereafter.

sification, so the head locks on within a few epochs and the remaining twenty epochs add only noise around the plateau. Training and validation accuracy track each other closely throughout, with no widening gap.



**Figure 11:** ViT confusion matrices. Errors concentrate on the dense/sparse-residential boundary in both panels; on the held-out set the single largest error mode is 4 `sparse_residential` images predicted as `dense_residential`.

The confusion matrices (Figure 11) repeat the pattern seen across all five models: the dense-versus-sparse residential boundary — a density judgement rather than a structural one — absorbs most of the mistakes, while `coastal_mansion` and `nursing_home` are essentially never confused with each other.

**No positive selection gap was observed.** Repeating the tuning-versus-held-out comparison made for the previous models (Table 19), the ViT sits at the far end of the capacity spectrum: with only 3,076 trainable parameters, macro- $F_1$  is actually *higher* on the held-out set than on the tuning split used to select the checkpoint (+0.010). This run therefore shows no positive tuning-to-held-out optimism for ViT. The direction is consistent with limited selection overfit, but one run on two small evaluation sets cannot establish that checkpoint-selection bias is zero. Section 7 discusses this observed gap alongside trainable capacity.

**Table 19:** ViT per-class  $F_1$  on the tuning split versus the held-out set.

| Class              | Tuning | Held-out | $\Delta$ |
|--------------------|--------|----------|----------|
| coastal_mansion    | 0.950  | 0.980    | +0.030   |
| nursing_home       | 0.971  | 0.980    | +0.009   |
| sparse_residential | 0.947  | 0.951    | +0.004   |
| dense_residential  | 0.952  | 0.951    | -0.001   |
| Macro- $F_1$       | 0.9549 | 0.9652   | +0.0103  |

## 5 Model Training and Tuning

### 5.1 Shared Training Protocol

Every final run in this report was produced by a single training entrypoint, `train.py`, rather than by five separate scripts. Each model is a module under `models/` that exposes one function, `build_model(num_classes, augmentation)`; the entrypoint injects the shared augmentation pipeline, builds the shared data splits, trains, selects a checkpoint, and writes results in a fixed schema. Keeping split construction, augmentation, and evaluation outside the per-model modules reduces accidental protocol drift and makes deviations visible in shared code and metadata. Table 20 lists the variables held constant across all five final runs.

**Table 20:** Controlled variables shared by all five final training runs.

| Variable         | Value                                                       |
|------------------|-------------------------------------------------------------|
| Random seed      | 42, with deterministic TensorFlow ops enabled               |
| Input            | $256 \times 256 \times 3$ , raw $[0, 255]$ RGB              |
| Data split       | identical stratified 85/15 train/tuning split (Section 3.1) |
| Augmentation     | shared policy of Section 3.3, training stream only          |
| Batch size       | 16                                                          |
| Epoch budget     | 32                                                          |
| Early stopping   | patience 20, monitoring tuning accuracy                     |
| Checkpointing    | best epoch by tuning accuracy, best weights restored        |
| Optimiser family | Adam                                                        |
| Loss             | categorical cross-entropy over one-hot labels               |
| Framework        | TensorFlow 2.21.0                                           |

Checkpoint selection and early stopping both monitor accuracy on the internal tuning split; the held-out set is reachable only through a separate loader behind an explicit `--evaluate-held-out`

flag, as described in Section 3.1. Repeatability is supported at two levels: the shared `set_seed()` seeds every random number generator and enables TensorFlow’s deterministic kernels, and every run writes a metadata record (`results/<run>.run.json`) capturing the git commit, seed, split counts including the excluded duplicate files, total and trainable parameter counts, best epoch, and TensorFlow version. Any number in this report can therefore be traced back to the exact code and data state that produced it.

Two per-model extension points keep the protocol from being a straitjacket. A model may expose `configure(dropout, learning_rate)` to accept head hyper-parameters from the command line, and `get_callbacks()` to add callbacks justified by that member’s own research — the custom CNN uses this hook for the `ReduceLROnPlateau` schedule specified by its source paper (Section 4.1). Neither hook can alter the data, the augmentation, or the evaluation.

## 5.2 Hyper-parameter Tuning

All hyper-parameter selection used the internal tuning split only; the held-out set was never consulted. The four transfer models share the same two-parameter head (dropout rate and learning rate), and all four final runs used dropout 0.3 and learning rate  $10^{-3}$ . These values were not simply assumed:

- **EfficientNet-B0** ran a one-factor-at-a-time experiment suite under a shortened budget (15 epochs, patience 3): the baseline (dropout 0.3, LR  $10^{-3}$ ) against a lower learning rate ( $3 \times 10^{-4}$ ) and against stronger dropout (0.4). Runs were ranked by tuning macro- $F_1$ , the baseline won, and only the winning configuration was retrained under the full 32-epoch protocol.
- **ViT** validated its choice of pretrained weights with the pretraining-source ablation of Section 4.5, again selected on the tuning split alone.
- **The custom CNN** takes its optimiser settings from its source paper rather than from a search, since faithful reproduction was the point of that baseline; the one free coefficient (L2 strength) is documented in Section 4.1.

This is deliberately light tuning. With frozen backbones the head has at most 8,196 trainable parameters, so the search space that matters is small, and we preferred spending the budget on a clean architecture comparison over per-model searches that would have confounded it.

## 5.3 Convergence

Because the epoch budget, batch size, seed, and data are identical across runs, convergence is compared directly in epochs — a measure independent of each member’s local machine — using the best-epoch column of Table 21 and the per-model curves in Section 4 (Figures 3, 4, and 10). The five runs span the full range of the budget. ViT locked on by epoch 10 — its frozen CLS embedding is already organised for linear classification, so the 3,076-parameter head has little left to learn — and was the only run to trigger early stopping (halting at epoch 30, best epoch 10 plus patience 20). The custom CNN and ResNet-50 peaked mid-schedule (epochs 24 and 25). MobileNetV2 and EfficientNet-B0 were still improving late: at best epochs 30 and 32

respectively, EfficientNet-B0’s checkpoint falls on the final epoch of its budget, which suggests a longer schedule could squeeze out slightly more and is noted as a limitation in Section 9. That the fastest- and slowest-converging models are both frozen-backbone transfer models with near-identical heads shows convergence speed here is a property of how linearly separable the pretrained embedding already is, not of trainable capacity.

## 6 Inference and Performance Evaluation

### 6.1 Evaluation Protocol

All reported metrics are produced by a single shared function, `evaluate()` in `shared/evaluate.py`, so every model is scored by identical code. For a given model and dataset it computes accuracy, per-class precision, recall, and  $F_1$  with supports, their macro averages, and the confusion matrix (via scikit-learn), and writes them to a fixed-schema JSON file plus a rendered confusion-matrix PNG under `results/`. The fixed schema is what makes the cross-model tables in this section possible without any manual transcription — every number below is read directly from those artifacts.

The held-out evaluation itself follows the protocol of Section 3.1: after training completes, the best checkpoint (selected on tuning accuracy) is restored and applied exactly once to the 400 sealed images of `val_23` (100 per class), loaded through the separate held-out loader behind the `--evaluate-held-out` flag. Augmentation layers are inactive at inference, so models are scored on clean images. Because the held-out set is exactly balanced, macro recall coincides with accuracy — a property visible in Table 21 that doubles as a sanity check on the pipeline.

### 6.2 Results

Table 21 ranks the five approaches on the held-out set.

**Table 21:** Held-out performance on `val_23` (400 images, 100 per class).

| Model           | Accuracy      | Macro P | Macro R | Macro F1      | Params | Best Epoch |
|-----------------|---------------|---------|---------|---------------|--------|------------|
| EfficientNet-B0 | <b>0.9775</b> | 0.9779  | 0.9775  | <b>0.9775</b> | 4.1M   | 32         |
| ResNet-50       | 0.9750        | 0.9757  | 0.9750  | 0.9750        | 23.6M  | 25         |
| ViT             | 0.9650        | 0.9659  | 0.9650  | 0.9652        | 85.8M  | 10         |
| MobileNetV2     | 0.9575        | 0.9592  | 0.9575  | 0.9576        | 2.3M   | 30         |
| Custom CNN      | 0.8450        | 0.8772  | 0.8450  | 0.8452        | 26.5M  | 24         |

Two facts dominate the leaderboard. First, the four transfer-learning models finish within 2.0 accuracy points of one another (0.9575–0.9775), while the from-scratch CNN trails the weakest of them by 11.25 points. The single largest factor in this study is therefore not the architecture but the presence of ImageNet pretraining. Second, the winner is also nearly the smallest model: EfficientNet-B0 reaches the best accuracy and macro- $F_1$  with 4.1M parameters, a twentieth of ViT’s 85.8M.

**Table 22:** Per-class  $F_1$  on the held-out set. The hardest class for every model is shown in bold.

| Class              | Custom CNN   | MobileNetV2  | ViT          | ResNet-50    | EfficientNet-B0 |
|--------------------|--------------|--------------|--------------|--------------|-----------------|
| coastal_mansion    | 0.990        | 0.975        | 0.980        | 0.995        | 0.985           |
| dense_residential  | <b>0.727</b> | 0.948        | 0.951        | <b>0.964</b> | <b>0.970</b>    |
| nursing_home       | 0.864        | <b>0.947</b> | 0.980        | 0.975        | 0.980           |
| sparse_residential | 0.800        | 0.960        | <b>0.950</b> | 0.966        | 0.976           |

**Per-class pattern.** The per-class breakdown (Table 22) is remarkably consistent with the predictions recorded during exploratory analysis. `coastal_mansion` is the easiest class for every model ( $F_1 \geq 0.975$  everywhere): the shoreline is a global, unambiguous cue, and three of the five models never predict this class for an image of any other class (perfect precision). `dense_residential` is the weakest class for the custom CNN, ResNet-50, and EfficientNet-B0, and within a point of the minimum for ViT and MobileNetV2 — for every model, the bottom of the table is a residential-density or institutional-layout judgement, never the shoreline class. The custom CNN’s failure is an extreme version of the same pattern rather than a different one — its `dense_residential` recall of 0.64 means 36 of 100 dense scenes were misread, almost all as `sparse_residential`, while every transfer model keeps dense-residential recall at 0.92 or above. This pattern is consistent with frozen ImageNet features providing useful mid-level texture and density information that the from-scratch model did not learn as effectively from 2,378 images. Because architecture and pretraining differ together in this comparison, the mechanism is not isolated directly.

**Error counts and statistical resolution.** In absolute terms the models misclassify 9 (EfficientNet-B0), 10 (ResNet-50), 14 (ViT), 17 (MobileNetV2), and 62 (custom CNN) of the 400 held-out images. The gap between the top two is a single image, and a 95% binomial confidence interval on 400 trials spans roughly  $\pm 1.5$  accuracy points at the top of the table (and  $\pm 3.5$  points at the custom CNN’s level). The ordering among the four transfer models is therefore suggestive rather than settled — all four intervals overlap — whereas the gap between any transfer model and the from-scratch baseline is far outside any interval and is the one clearly separated result in this evaluation. This caveat is carried into the recommendation in Section 9.

Confusion matrices for each model, and the per-model diagnosis of where the errors sit, are given alongside the corresponding architecture in Section 4; the cross-model view of those errors is taken up in Section 7.

## 7 Strengths, Weaknesses, and Comparative Insights

### 7.1 Per-Approach Assessment

**Custom CNN (Pyramidal Net).** Its strength is exactly its role: as the only model with no external weights, it isolates what our 2,378 images alone can teach, and on classes with a global, scene-level cue it is genuinely competitive ( $F_1 = 0.990$  on `coastal_mansion`, with perfect precision). Its weaknesses follow from its capacity being in the wrong place: 97% of its 26.5M parameters sit in a single dense layer fitted to a few thousand images, producing the largest

train-validation gap of the study, the worst held-out accuracy (0.8450), a collapse on the density boundary (dense-residential recall 0.64), and a 6.3-point gap between tuning and held-out macro- $F_1$  — the least trustworthy validation signal of the five. It is also the only model whose early training was unstable (Figure 3).

**ResNet-50.** The strong default lived up to its reputation: 0.9750 accuracy, no class below  $F_1 = 0.964$ , smooth convergence, and a small observed tuning-to-held-out gap ( $-0.006$  macro- $F_1$ ). Its weakness is cost-effectiveness rather than quality: it needs 23.6M parameters to reach an accuracy within one image of EfficientNet-B0, which uses roughly a sixth as many parameters. As a deployment artifact it is the largest CNN checkpoint in the study for no measurable accuracy benefit.

**EfficientNet-B0.** The best result of the study — 0.9775 accuracy and macro- $F_1$  with only 4.1M total parameters and 5,124 trainable — and the only model to hold every class at  $F_1 \geq 0.970$ . Compound scaling’s accuracy-per-parameter promise [12] is borne out at the smallest scale of the family. Its hyper-parameters were also the most systematically chosen, via a one-factor-at-a-time suite (Section 5). Two caveats: its best epoch was the last epoch of the budget, so it is plausibly slightly undertrained, and its margin over ResNet-50 is one image — a tie within noise, not a demonstrated superiority.

**MobileNetV2.** MobileNetV2 represents the efficiency-oriented extreme: 2.3M parameters — 2.6% of ViT’s footprint — still delivers 0.9575 accuracy, making it the best accuracy-per-parameter in the study and the natural choice if the classifier had to run on an edge device such as a drone or embedded mapping unit [9]. Its weakness is that it is the least accurate of the transfer models, and its error signature differs from the others: its largest single confusion is `dense_residential`  $\rightarrow$  `nursing_home` (7 images) rather than the dense  $\rightarrow$  sparse confusion that dominates elsewhere. This pattern may indicate that its aggressively factorised representation is less effective at separating the building-footprint cues that distinguish institutional from residential blocks, but no feature-level ablation was run.

**Vision Transformer (ViT-B/16).** The structural outlier earns its place in the comparison: it converged fastest (best epoch 10), and its held-out macro- $F_1$  was 0.010 above its tuning value, so no positive tuning-to-held-out optimism was observed in this run. It also showed that frozen global self-attention can transfer to this aerial-imagery task. But at this dataset scale its costs are not repaid: 85.8M parameters — by far the heaviest training and inference footprint of the study — for an accuracy 1.0 point *below* ResNet-50. In Pareto terms ResNet-50 dominates it on both size and accuracy. The literature’s caution that transformers typically require more data [10] is thus refined by our result: a frozen ViT is perfectly *usable* at 2,378 images, but nothing about the task rewarded its extra capacity.

## 7.2 Accuracy versus Cost

**Table 23:** Consolidated comparison.  $\Delta F_1$  is the observed held-out minus tuning macro- $F_1$  gap; a negative value means tuning performance was higher.

| Model           | Total params | Trainable  | Held-out acc. | Macro $F_1$   | $\Delta F_1$ |
|-----------------|--------------|------------|---------------|---------------|--------------|
| EfficientNet-B0 | 4,054,695    | 5,124      | <b>0.9775</b> | <b>0.9775</b> | +0.0084      |
| ResNet-50       | 23,595,908   | 8,196      | 0.9750        | 0.9750        | −0.0059      |
| ViT-B/16        | 85,801,732   | 3,076      | 0.9650        | 0.9652        | +0.0103      |
| MobileNetV2     | 2,263,108    | 5,124      | 0.9575        | 0.9576        | +0.0147      |
| Custom CNN      | 26,512,212   | 26,511,444 | 0.8450        | 0.8452        | −0.0627      |

Plotting accuracy against parameter count, only two models sit on the Pareto frontier: MobileNetV2 (smallest) and EfficientNet-B0 (most accurate). ResNet-50 is dominated by EfficientNet-B0 — more parameters with a lower observed accuracy — and ViT is dominated by ResNet-50 on both axes. The custom CNN is dominated by everything: more parameters than three of the four transfer models and 11–13 points less accuracy. For deployment scenarios in this study’s range, the practical choices are therefore EfficientNet-B0 by default and MobileNetV2 when the size budget is severe.

## 7.3 Data Efficiency: What Pretraining Buys

The comparison quantifies transfer learning’s value on this task with unusual directness, because everything except pretraining is controlled. Fitting 3,076–8,196 head parameters on frozen ImageNet features beats fitting 26.5M parameters from scratch by 11.25–13.25 accuracy points, at a fraction of the training cost. Moreover, the deficit is concentrated precisely where the task is fine-grained: the from-scratch model matches the transfer models on *coastal\_mansion* (a coarse, colour-and-shoreline cue) and loses massively on the density classes (Table 22). This is the boundary sketched in Section 4.1: from-scratch CNNs suffice for coarse scene vocabulary, while fine-grained spatial-arrangement distinctions demand either far more data or borrowed mid-level features. Our result thus lands on the same side as Kwenda et al. [8] and the review consensus [10], but with the confound of differing pipelines removed.

## 7.4 Capacity and the Trustworthiness of Validation Metrics

The  $\Delta F_1$  column of Table 23 completes the analysis threaded through Section 4. Checkpoint selection maximises tuning accuracy, so a negative gap is consistent with selection optimism. The custom CNN shows a −6.3-point gap and ResNet-50 a −0.6-point gap; for models with 3,076–5,124 trainable parameters the sign reverses and held-out performance slightly exceeds tuning. These observations are consistent with reduced selection overfit when only a small head is trained, but a single run and two small evaluation sets do not identify the true bias or distinguish it from sampling variability. The practical implication is therefore cautious: freezing the backbone can make validation-based selection more stable, while validation scores from high-capacity models warrant greater scrutiny.

## 7.5 Error Overlap and Diversity

Do the five models fail on the same images? At the class level, largely yes: every confusion matrix concentrates its mass on the dense/sparse-residential boundary, and no model ever confuses `coastal_mansion` with `nursing_home` in either direction — the two classes with distinctive global structure. The custom CNN’s errors look like a magnified superset of the transfer models’ errors rather than a different failure mode.

Within the transfer models, however, the *direction* of error differs instructively. ResNet-50 and EfficientNet-B0 spread their few errors thinly with a slight dense→sparse lean; ViT’s largest single cell is the reverse, sparse→dense (4 images); MobileNetV2’s is dense→nursing-home (7 images). The top models are therefore not making identical mistakes, which has a concrete consequence: a majority vote or probability-averaging ensemble of ResNet-50, EfficientNet-B0, and ViT would plausibly recover several of the 9–14 residual errors each makes alone. We flag this as future work (Section 12) rather than reporting it, because building an ensemble after seeing held-out results would itself constitute test-set leakage under our protocol.

## 8 Deployment via Containerisation

A model that exists only as a checkpoint in a researcher’s working directory is not usable by anyone else. The final stage of this project therefore packages the best-performing classifier as a self-contained container image that accepts an aerial image through a web interface and returns a predicted category with per-class softmax scores.

### 8.1 Deployed Model

The deployed classifier is **EfficientNet-B0**. Every architecture and checkpoint was fixed using the internal tuning split before the five frozen models were evaluated once on held-out data. EfficientNet-B0 then produced the highest held-out macro- $F_1$  (0.9775, Section 6.2) and was packaged as the assignment’s best-performing demonstrator. This post-evaluation packaging decision did not feed back into training, hyper-parameter selection, or another held-out run. It is also a practical choice independently of the one-image lead over ResNet-50: at 4.05M parameters its checkpoint is roughly an order of magnitude smaller than the custom CNN’s 304 MB, which materially affects image size, registry push time, and container start-up.

### 8.2 Container Architecture

The image is built from `python:3.12-slim` and contains four layers of interest, ordered so that the expensive dependency installation is cached and only the cheap application layers rebuild when code changes (Table 24).

**Table 24:** Container image composition.

| Layer        | Contents                                      | Purpose                                                |
|--------------|-----------------------------------------------|--------------------------------------------------------|
| Base         | <code>python:3.12-slim</code>                 | minimal Python runtime                                 |
| Dependencies | <code>frontend/requirements.txt</code>        | Streamlit, TensorFlow-CPU, Pillow, NumPy               |
| Shared code  | <code>shared/</code>                          | class names and input size from <code>config.py</code> |
| Application  | <code>frontend/web.py</code>                  | Streamlit inference interface                          |
| Model        | <code>results/&lt;model&gt;_best.keras</code> | trained checkpoint                                     |

The container exposes port 8501 and declares a `HEALTHCHECK` that polls Streamlit's `/_stcore/health` endpoint, so an orchestrator can distinguish a started container from a genuinely ready one. The entrypoint runs Streamlit bound to `0.0.0.0` so the service is reachable from outside the container.

### 8.3 Inference Pipeline

At request time the application performs four steps: it loads the uploaded file and converts it to RGB, resizes it to the `IMAGE.SIZE` imported from `shared/config.py`, runs a single forward pass, and renders the result as a predicted label with its maximum softmax score plus a bar chart of the full softmax distribution over all four classes. Presenting the complete distribution rather than only the argmax is deliberate: on a fine-grained problem where `dense_residential` and `sparse_residential` are genuinely confusable, a user benefits from seeing that a prediction was made at 0.55 against 0.40 rather than at 0.99.

The model is loaded once and cached with Streamlit's `@st.cache_resource` decorator, so the checkpoint is deserialised on first request rather than on every upload. Loading is wrapped in exception handling: if the checkpoint cannot be read — most plausibly a Keras version mismatch — the interface starts in a clearly labelled preview mode and displays the underlying error rather than failing silently or crashing.

### 8.4 Design Decisions

Three choices are worth justifying.

**Preprocessing travels with the model.** Augmentation and any normalisation are layers *inside* the saved Keras graph rather than steps in a separate inference script. Reusing graph-level normalisation at inference reduces the risk of incorrectly scaled inputs and training-serving skew; the container therefore needs no separate normalisation implementation.

**The container imports the same configuration module as training.** Class names and input resolution are read from `shared/config.py`, the same configuration module used to produce the checkpoint. This shared source, together with the deployment contract test, makes label-order drift detectable and avoids a second hard-coded class list in the application.

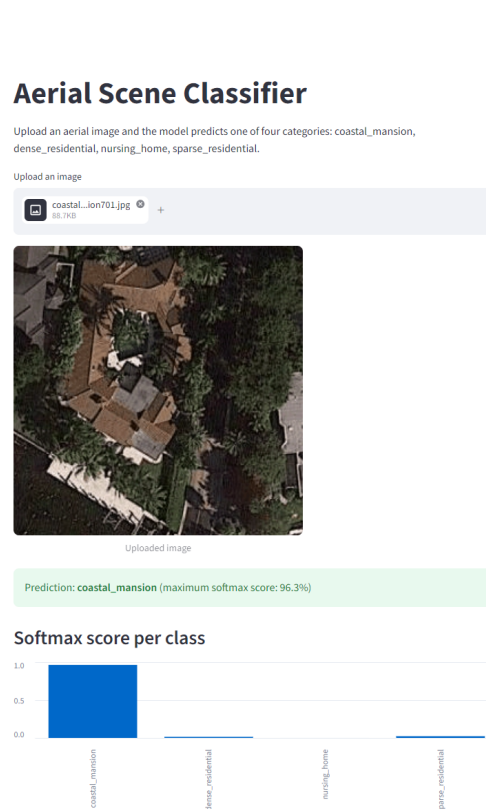
**CPU-only TensorFlow.** The image pins `tensorflow-cpu` rather than the full distribution. Inference on a single image does not benefit meaningfully from a GPU, and the CPU build avoids bundling CUDA libraries that would inflate the image substantially and require GPU-enabled hosts.

## 8.5 Build and Deployment

The image is built and published with:

```
docker build -t <username>/<repository>:<version> .
docker push <username>/<repository>:<version>
docker run --name <container> -p 8501:8501 <username>/<repository>:<version>
```

The interface is then available at `http://localhost:8501`. Because trained checkpoints exceed the file-size limits of our version-control host, \*.keras artifacts are excluded from the repository and distributed through the container image instead; a .dockerignore allow-list admits only the single checkpoint being served, keeping the other models' weights out of the build context.



**Figure 12:** Running Streamlit interface in the Docker container. A held-out coastal mansion image is classified correctly with a maximum softmax score of 96.3%, and the full four-class probability distribution is displayed below the prediction.

## 8.6 Verification and Limitations

Deployment was verified by building the image, starting the container, receiving HTTP 200 from its health endpoint, and submitting the held-out image shown in Figure 12. The running service loaded the selected EfficientNet-B0 checkpoint and returned coastal\_mansion with a maximum

softmax score of 96.3%. The repository also includes automated contract tests (`tests/`) for the shared data and training pipelines plus the deployed model path, checkpoint deserialisation, image preprocessing, output shape, and probability normalisation. These tests establish that the data split is deterministic and disjoint, that the duplicate-label conflict is excluded, and that the model served by Docker follows the same raw-input and fixed-class contracts used during evaluation.

Two limitations should be acknowledged. First, the interface itself is not covered by automated tests — verification is manual, through the running container. Second, the service is a single-container demonstrator: it performs no batching, authentication, request logging, or horizontal scaling, and would require an orchestration layer and a proper inference server before production use. These are deliberate scope boundaries rather than oversights, given that the coursework requirement is a working containerised demonstrator.

## 9 Results, Insights, and Discussion

**What this benchmark says about fine-grained aerial classification.** The headline finding is that four visually confusable residential categories — the deliberately fine-grained partition PatternNet was designed to produce [14] — are solvable to 96–98% accuracy by frozen ImageNet features with a head of a few thousand parameters. No fine-tuning, no task-specific architecture, and no aggressive hyper-parameter search were needed to reach that level. The genuinely scarce resource in this regime is not model capacity but labelled data, and pretraining is how capacity is converted into something 2,378 images can steer: the same experiment run without pretraining (our custom CNN) lands 13 macro- $F_1$  points lower despite having more parameters than three of the four transfer models. Architecture choice *among* pretrained backbones, by contrast, is worth at most 2 points on this task — second-order compared with the decision to transfer at all.

**Where the remaining errors live.** Every model’s residual errors concentrate on the dense-versus-sparse residential boundary, exactly as the exploratory analysis predicted before any training. We read this less as a model deficiency than as a property of the label space: density is a continuous quantity that the benchmark discretises into two names, so scenes near the threshold are inherently ambiguous — the dataset itself contains one image assigned to two classes (Section 2.5), direct evidence that even the annotation process struggled at this boundary. Accuracy meaningfully above 98% on this partition would likely require modelling that ambiguity (soft labels, ordinal density regression) rather than a better classifier.

**Final model recommendation.** We recommend **EfficientNet-B0** as the final model. It tops the leaderboard (0.9775 accuracy and macro- $F_1$ ), holds every class at  $F_1 \geq 0.970$ , and does so with the second-smallest parameter count and the smallest checkpoint among the top models. MobileNetV2 is 45% smaller, but trails by 2 accuracy points. We state the main caveat plainly: EfficientNet-B0’s margin over ResNet-50 is one image on a 400-image test set, so this evaluation does not resolve their accuracy ordering. The recommendation therefore rests on EfficientNet-B0’s comparable observed accuracy at substantially lower cost, not on proven statistical superi-

ority. If the deployment budget were tightened to embedded hardware, MobileNetV2 would be the rational trade.

**Limitations.** Four limitations bound our claims. (i) Every result is a single run at seed 42; without repetition across seeds we cannot attach variances. Fixed seeds, deterministic operations, and recorded metadata improve repeatability within the pinned environment, but exact cross-machine equivalence was not experimentally established. (ii) The 400-image test set implies roughly  $\pm 1.5$ -point confidence intervals at the top of the leaderboard, so the ordering among the four transfer models is not statistically resolved — only the transfer-versus-scratch gap is. (iii) All four backbones were fully frozen. This was a deliberate controlled variable, but it means we measured pretrained *representations*, not the ceiling of each architecture under fine-tuning; EfficientNet-B0’s best epoch landing on the final epoch of its budget likewise hints at headroom we did not spend. (iv) The augmentation policy, input resolution, and other shared settings were fixed for comparability rather than tuned per model, so each individual model is likely slightly below its own optimum — the price of a fair comparison, paid knowingly.

## 10 Individual Contributions

Each member owned one deep learning approach end-to-end — implementation under the shared `build_model` contract, experiments, final training run, results artifacts, figures, and the corresponding report subsection — developed on a dedicated branch and merged after review. Table 25 summarises the ownership; the paragraphs below record each member’s wider contributions to the shared infrastructure, analysis, deployment, and report.

**Table 25:** Model ownership and development branches.

| Member              | Approach owned                      | Branch                       |
|---------------------|-------------------------------------|------------------------------|
| Pang Jia De         | Custom CNN — from-scratch baseline  | <code>model/customcnn</code> |
| Daryl Chong Han Kin | ResNet-50 — transfer learning       | <code>model/ResNet-50</code> |
| Tan Jian Xin        | MobileNetV2 — transfer learning     | <code>mobilenetv2</code>     |
| Kaiser Yee          | Vision Transformer (ViT-B/16)       | <code>model/vit</code>       |
| Jun Kiat            | EfficientNet-B0 — transfer learning | <code>model/effNet-b0</code> |

**Pang Jia De** implemented and trained the from-scratch Pyramidal Net baseline, reproducing the source paper’s architecture and training configuration and documenting the deviations (Section 4.1). He also led the exploratory data analysis (`eda.ipynb`), including the discovery of the byte-identical image pair assigned to conflicting classes that led to the shared exclusion mechanism, drove the decision to standardise inputs at the native  $256 \times 256$  resolution, and maintained shared project configuration and the Docker build files through successive revisions.

**Daryl Chong Han Kin** implemented and trained the ResNet-50 transfer model, including the tuning-versus-held-out bias analysis that became the template for the capacity–trustworthiness comparison across all five models (Section 7). He also acted as report integrator: maintaining the LaTeX sources, merging members’ section drafts, and editing the report for a consistent structure and voice.

**Tan Jian Xin** implemented and trained the MobileNetV2 transfer model, the efficiency-oriented end of the comparison, and contributed the accuracy-per-parameter analysis of the deployment-constrained design point.

**Kaiser Yee** implemented and trained the ViT-B/16 transfer model, including the input-resize fix required by ViT’s fixed position-embedding table — the one genuine integration bug of the project — and designed and ran the pretraining-source ablation (ImageNet-21k versus 1k-finetuned weights), the study’s cleanest example of an experimentally falsified assumption.

**Jun Kiat** implemented and trained the EfficientNet-B0 model that ultimately topped the leaderboard. He also built much of the shared engineering scaffolding: the automated contract tests under `tests/`, the WSL2 GPU environment scripts, the held-out evaluation script, the one-factor-at-a-time experiment suite and its summary tooling, and an initial containerised inference service.

## 11 Reflection

### 11.1 Evaluation Discipline Is a Design Problem, Not a Willpower Problem

The lecture-level statement “do not tune on your test set” sounds like an instruction to be careful. The lasting lesson of this project is that carefulness does not scale across five people and dozens of runs — the protection has to be built into the code. We designed the pipeline so that the held-out loader is a separate function behind an explicit flag, never passed to `model.fit` and never visible to a callback. This separation makes accidental leakage less likely and easier to audit, although the explicit evaluation flag means the protocol still depends on disciplined use. The payoff was concrete. Our custom CNN’s tuning macro- $F_1$  of 0.908 overstated its true held-out 0.845 by more than six points, with the entire gap concentrated in one class. Had we reported tuning metrics — a natural mistake, since the model never received gradients from those images — we would have published the wrong number *and* the wrong conclusion about which class was solved. Watching that gap shrink to zero as trainable capacity fell across the other four models turned an abstract warning about validation optimism into something we measured ourselves. None of us will again read a validation score without asking how much capacity the model had to fit the validation set’s idiosyncrasies.

### 11.2 A Controlled Comparison Is a Team Contract

We set out to compare architectures, and learnt that the comparison is only as good as the discipline around it. Fixing the seed, split, input size, batch size, epoch budget, augmentation policy,

and evaluation code across five members meant freezing shared/ early and treating changes to it as team-wide events, because any silent change would invalidate everyone's prior runs. The fixed result schema was the quiet hero: because every run wrote the same JSON structure, cross-model tables assembled themselves without transcription, and each run's metadata (git commit, seed, split counts, parameter counts, best epoch) made every number in this report auditable months later.

We also learnt when to renegotiate the contract. The pipeline originally used  $224 \times 224$  inputs; the EDA showed this discarded 23% of native pixels, and changing it was cheap precisely because we caught it before anyone had results to invalidate. The general lesson: agree the controlled variables before training starts, change them only by consensus, and instrument everything so that each reported number remains traceable to recorded configuration and metadata.

### 11.3 Transfer Learning in a Low-Data Regime

Three findings reshaped our intuitions. First, the sheer size of the pretraining effect: heads of 3,076–8,196 parameters on frozen ImageNet features beat 26.5M parameters trained from scratch by 11–13 points, and the deficit concentrated exactly on the fine-grained density distinctions — a pattern consistent with pretrained representations providing useful mid-level texture information that the from-scratch model did not learn as effectively. Second, published orderings do not transfer across task granularity: the source paper behind our custom CNN reports it competitive with pretrained networks on semantically disjoint categories, and that conclusion simply did not survive contact with four confusable residential classes. Reproducing a paper faithfully and then explaining the divergence taught us more than either step alone. Third, assumptions are cheap to test and often wrong: we expected raw ImageNet-21k features to transfer better to aerial imagery than 1k-finetuned ones, ran the ablation, and were cleanly refuted — in a frozen-backbone regime, the linear separability that supervised finetuning imposes matters more than feature generality.

### 11.4 Engineering Lessons from the Debugging Trail

The bugs we hit were more instructive than the code that worked first time. The ViT input-size failure was the sharpest: CNN backbones silently accept  $256 \times 256$  inputs because convolution weights are position-agnostic, but ViT's fixed position-embedding table made the same mismatch a hard error, forcing us to understand *why* the two families differ rather than just resizing and moving on. The augmentation-ordering bug was subtler: placing rescaling before the brightness layer — which is configured for a  $[0, 255]$  range — silently saturated images to black, a reminder that numerically valid pipelines can still be semantically wrong, and that augmented batches should be visualised, not assumed. An early  $\pm 50\%$  photometric jitter that destroyed the very texture cues our EDA said the task depends on taught the same inspect-your-data lesson from the other side.

Environment work was its own education: getting TensorFlow's GPU path working under WSL2 and pinning Python and TensorFlow versions across five machines. Deterministic settings, identical split rules, and recorded metadata reduced the risk of unnoticed differences between five development environments, while hardware and dependency variations could still affect exact numerical reproduction. The automated contract tests earned their keep repeatedly, catching

regressions in the split logic before they could corrupt anyone’s run. If we started again, we would write those tests first, not midway.

### 11.5 What We Would Do Differently

Beyond starting with the tests, we would budget training time by architecture rather than uniformly — EfficientNet-B0’s best epoch landing on its final epoch shows a flat 32-epoch cap does not fit every model equally. We would also schedule multiple seeds from the outset; single-run results were the right trade under our compute constraints, but the confidence-interval analysis of Section 6.2 would have been stronger as an empirical variance. Both are carried into Section 12.

## 12 Future Work

**Partial fine-tuning with discriminative learning rates.** The most direct extension is to unfreeze the top blocks of the winning backbones with a learning rate one to two orders of magnitude below the head’s, keeping earlier layers frozen. Our frozen-backbone protocol measured pretrained representations, not each architecture’s ceiling, and two signs point to headroom: EfficientNet-B0’s best epoch fell on the last epoch of its budget, and the residual errors are fine-grained texture judgements that adapting the top convolutional blocks to aerial imagery may help address. The selection discipline must carry over: any fine-tuning experiment would be ranked on the internal tuning split, with the held-out set touched once at the end.

**Ensembling and test-time augmentation.** Section 7 showed the top models err in different directions (ResNet-50 and EfficientNet-B0 lean dense→sparse, ViT the reverse, MobileNetV2 toward nursing\_home), which is precisely the diversity an averaging ensemble exploits; with only 9–14 errors per model, recovering even a handful would be a meaningful relative gain. Test-time augmentation is a natural companion on aerial data: since flips and rotations are label-preserving here, averaging predictions over the dihedral group costs only inference time and no retraining. Both must be designed *before* a fresh evaluation — assembling an ensemble to fix errors already observed on the held-out set would be selection on the test set.

**Multiple seeds and statistical testing.** Rerunning the five final configurations over several seeds would replace our single-run leaderboard with means and variances, and paired tests (e.g. McNemar’s on the 400 held-out predictions) could establish whether EfficientNet-B0’s one-image margin over ResNet-50 ever becomes a real ordering. This is the least computationally expensive way to strengthen the study’s weakest statistical point.

**Interpretability of the density decision.** Grad-CAM on the convolutional models and attention-map visualisation on ViT would test our standing interpretation directly: that coastal\_mansion is recognised by shoreline, and that dense/sparse errors occur on scenes near the density threshold. Confirming that the models attend to buildings and spacing — rather than to spurious cues such as image brightness — would also strengthen the case for deploying the classifier on imagery from other regions.

**Calibration and abstention.** The inference interface reports uncalibrated softmax scores; we did not test whether a score of 0.9 corresponds to 90% empirical correctness. Measuring calibration error on held-out predictions, applying temperature scaling if needed, and adding an “uncertain — refer to a human” band for borderline density cases would make the score operationally meaningful, which matters for the mapping and planning applications that motivated the task.

**Hardened serving path.** The Streamlit UI suits demonstration; a production path would add a FastAPI REST endpoint alongside it (machine-to-machine JSON inference), container build-and-health checks in CI, and a quantised or TensorFlow-Lite export — most naturally of MobileNetV2 — for on-device inference, subject to verifying metric parity with the trained checkpoint.

**Scaling beyond the partition.** Finally, the four-class study invites two outward steps: training on more (eventually all 38) PatternNet categories to test whether the frozen-backbone approach holds as the label space grows, and evaluating cross-dataset transfer (e.g. on AID or NWPU-RESISC45 residential classes) to measure the domain gap our single-source benchmark cannot reveal. Remote-sensing-specific pretrained backbones would be the natural comparison point there, substituting in-domain pretraining for ImageNet’s.

## References

- [1] Ahmed M. Abdelmonem, Mohamed Maher Ata, Abdelhamied A. Atey, A. A. Shaalan, and Rania A. El-Sayed. High-performance scene classification in remote sensing imagery using a custom deep CNN architecture. *Scientific Reports*, 16:5946, 2026. pages 14, 15, 16, 18, 19
- [2] Yakoub Bazi, Laila Bashmal, Mohamad M. Al Rahhal, Reham Al Dayil, and Naif Al Ajlan. Vision transformers for remote sensing image classification. *Remote Sensing*, 13(3):516, 2021. pages 28
- [3] Gong Cheng, Xingxing Xie, Junwei Han, Lei Guo, and Gui-Song Xia. Remote sensing image scene classification meets deep learning: Challenges, methods, benchmarks, and opportunities. *IEEE Journal of Selected Topics in Applied Earth Observations and Remote Sensing*, 13:3735–3756, 2020. pages 4, 6
- [4] Alexey Dosovitskiy, Lucas Beyer, Alexander Kolesnikov, Dirk Weissenborn, Xiaohua Zhai, Thomas Unterthiner, Mostafa Dehghani, Matthias Minderer, Georg Heigold, Sylvain Gelly, Jakob Uszkoreit, and Neil Houlsby. An image is worth 16x16 words: Transformers for image recognition at scale. In *International Conference on Learning Representations (ICLR)*, 2021. pages 7, 27, 28
- [5] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 770–778, 2016. pages 7, 19
- [6] Simon Kornblith, Jonathon Shlens, and Quoc V. Le. Do better ImageNet models transfer better? In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 2661–2671, 2019. pages 28, 29
- [7] Alex Krizhevsky, Ilya Sutskever, and Geoffrey E. Hinton. ImageNet classification with deep convolutional neural networks. In *Advances in Neural Information Processing Systems 25*, pages 1097–1105, 2012. pages 4, 6
- [8] Clopas Kwenda, Mandlenkosi Gwetu, and Jean Vincent Fonou-Dombeu. Hybridizing deep neural networks and machine learning models for aerial satellite forest image segmentation. *Journal of Imaging*, 10(6):132, 2024. pages 6, 7, 36
- [9] Mark Sandler, Andrew Howard, Menglong Zhu, Andrey Zhmoginov, and Liang-Chieh Chen. MobileNetV2: Inverted residuals and linear bottlenecks. In *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 4510–4520, 2018. pages 7, 24, 25, 35
- [10] Hashibul Ahsan Shoaib, Hadiur Rahman Nabil, Md Anisur Rahman, Md Mohsin Kabir, M. F. Mridha, and Jungpil Shin. Advancements and challenges of deep learning architectures for aerial image analysis: A systematic review. *Intelligent Systems with Applications*, 27:200537, 2025. pages 7, 35, 36

- [11] Andreas Steiner, Alexander Kolesnikov, Xiaohua Zhai, Ross Wightman, Jakob Uszkoreit, and Lucas Beyer. How to train your ViT? Data, Augmentation, and Regularization in Vision Transformers. *Transactions on Machine Learning Research*, 2022. arXiv:2106.10270. pages 29
- [12] Mingxing Tan and Quoc V. Le. EfficientNet: Rethinking model scaling for convolutional neural networks. In *Proceedings of the 36th International Conference on Machine Learning (ICML)*, volume 97, pages 6105–6114, 2019. pages 7, 21, 22, 23, 35
- [13] Hugo Touvron, Matthieu Cord, Matthijs Douze, Francisco Massa, Alexandre Sablayrolles, and Hervé Jégou. Training data-efficient image transformers & distillation through attention. In *International Conference on Machine Learning (ICML)*, pages 10347–10357, 2021. pages 28
- [14] Weixun Zhou, Shawn Newsam, Congmin Li, and Zhenfeng Shao. PatternNet: A benchmark dataset for performance evaluation of remote sensing image retrieval. *ISPRS Journal of Photogrammetry and Remote Sensing*, 145:197–209, 2018. pages 6, 40
- [15] Xiao Xiang Zhu, Devis Tuia, Lichao Mou, Gui-Song Xia, Liangpei Zhang, Feng Xu, and Friedrich Fraundorfer. Deep learning in remote sensing: A comprehensive review and list of resources. *IEEE Geoscience and Remote Sensing Magazine*, 5(4):8–36, 2017. pages 4, 6

## A Supplementary Material

### A.1 Consolidated Hyper-parameters

**Table 26:** Final-run hyper-parameters for all five approaches. Shared protocol settings (seed 42, batch 16, 32-epoch budget, patience 20, input  $256 \times 256$ ) are listed in Table 20 and omitted here.

| Model           | Learning rate        | Dropout | Regularisation / callbacks             | Trainable params |
|-----------------|----------------------|---------|----------------------------------------|------------------|
| Custom CNN      | $2.5 \times 10^{-4}$ | none    | L2 $10^{-4}$ (conv); ReduceLROnPlateau | 26,511,444       |
| ResNet-50       | $10^{-3}$            | 0.3     | frozen backbone                        | 8,196            |
| EfficientNet-B0 | $10^{-3}$            | 0.3     | frozen backbone                        | 5,124            |
| MobileNetV2     | $10^{-3}$            | 0.3     | frozen backbone                        | 5,124            |
| ViT-B/16        | $10^{-3}$            | 0.3     | frozen backbone                        | 3,076            |

### A.2 Repository Structure

```
CSC3109-Machine-Learning/
|- dataset/set 23/      provided training data (700 images x 4 classes)
|- dataset/val 23/     sealed held-out set (100 images x 4 classes)
|- shared/             frozen team contract: config, data splits,
|                       augmentation policy, evaluate()
|- models/<name>/      one module per approach exposing
|                       build_model(num_classes, augmentation)
```

|                   |                                         |
|-------------------|-----------------------------------------|
| - train.py        | shared training/evaluation entrypoint   |
| - tests/          | pytest contract tests (data + training) |
| - results/        | per-run JSON metrics, metadata, curves, |
|                   | confusion matrices, checkpoints         |
| - frontend/web.py | Streamlit inference UI                  |
| - Dockerfile      | deployment image (python:3.12-slim)     |
| - eda.ipynb       | exploratory data analysis               |
| - docs/           | this report                             |

### A.3 Reproduction Instructions

All results in this report can be regenerated from the repository root. The dataset directories are expected under `dataset/` (override with the `CSC3109_DATA_DIR` environment variable).

```
# environment
pip install -r requirements.txt

# verify the shared-pipeline contract
pytest

# reproduce a final run (identical form for custom_cnn,
# resnet50, efficientnet_b0, mobilenetv2, ViT)
python train.py --model resnet50 --run-name resnet50 \
    --epochs 32 --patience 20 --batch-size 16 \
    --run-type final --evaluate-held-out
```

Each run writes `results/<run>_run.json` (metadata: seed, git commit, split counts, parameters, best epoch), `<run>_history.json` (per-epoch curves), `<run>_tuning.json` and `<run>.json` (fixed-schema metrics on the tuning and held-out sets), and the corresponding confusion-matrix and training-curve figures. The shared `set_seed()` seeds the random number generators and requests deterministic TensorFlow kernels. This supports repeatability under the recorded environment, but exact numerical equivalence across reruns or machines is not claimed; TensorFlow 2.21.0 was used for all final runs.